Algoritmi Strutture Dati e Complessità

Casolino Ilaria, Giamberardini Matteo

A.A. 2023/2024

Indice

1 Algoritmi **5**
- 1.1 Il problema dell'ordinamento . . . 5
- 1.2 Costo di un algoritmo . . . 5
  - 1.2.1 Definizione . . . 5
  - 1.2.2 Complessità temporale . . . 6
  - 1.2.3 Complessità spaziale . . . 6
  - 1.2.4 Istruzione dominante . . . 6
  - 1.2.5 Regola di Horner . . . 6
- 1.3 Andamento asintotico . . . 6
  - 1.3.1 Casi da ricordare . . . 7
  - 1.3.2 Problemi . . . 8

2 Tecniche di progettazione degli algoritmi **9**
- 2.1 La programmazione dinamica . . . 9
  - 2.1.1 Problema delle aste . . . 9
  - 2.1.2 Moltiplicazione di Matrici . . . 11
  - 2.1.3 Conclusione . . . 11
- 2.2 Algoritmi golosi . . . 12
  - 2.2.1 Problema della selezione di attività . . . 12
  - 2.2.2 Procedimento per sviluppare un algoritmo avido o goloso . . . 13
  - 2.2.3 Elementi della strategia avida . . . 13
  - 2.2.4 Algoritmo di Huffman . . . 14

3 Algoritmi di ordinamento (sorting) **15**
- 3.1 Insertion Sort . . . 15
  - 3.1.1 Codice C iterativo InsertionSort . . . 15
  - 3.1.2 Esempio iterativo InsertionSort . . . 16
  - 3.1.3 Codice C ricorsivo InsertionSort . . . 16
  - 3.1.4 Esempio ricorsivo InsertionSort . . . 16
  - 3.1.5 InsertionSort: Analisi dei costi . . . 17
  - 3.1.6 Equazione di ricorrenza . . . 17
  - 3.1.7 Tecnica dello srotolamento . . . 18
  - 3.1.8 Analisi dei costi di confronto dell'InsertionSort . . . 18
  - 3.1.9 Analisi dei costi di assegnazione dell'InsertionSort . . . 19
  - 3.1.10 Analisi dei costi di memoria dell'InsertionSort . . . 20
- 3.2 MergeSort . . . 20
  - 3.2.1 Metodo Divide et Impera . . . 20
  - 3.2.2 MergeSort: equazione di ricorrenza . . . 20
  - 3.2.3 PseudoCodice MergeSort . . . 20
  - 3.2.4 Codice C MergeSort . . . 22
  - 3.2.5 Esempio MergeSort . . . 23
- 3.3 QuickSort . . . 23
  - 3.3.1 Analisi dei costi del QuickSort . . . 24
  - 3.3.2 Equazione di ricorrenza QuickSort . . . 24
  - 3.3.3 Pseudo Codice QuickSort . . . 24
  - 3.3.4 Codice C QuickSort . . . 25
  - 3.3.5 Invariante di ciclo QuickSort . . . 25
- 3.4 SelectionSort . . . 26
  - 3.4.1 Codice C SelectionSort . . . 26

3.4.2 Analisi dei costi SelectionSort . . . 27
3.5 CountingSort . . . 27
3.6 BucketSort . . . 28
3.7 HeapSort . . . 28
3.7.1 Definizione di Heap . . . 28
3.7.2 Code di max-priorità . . . 31

4 I Grafi **33**
4.1 Definizioni dei Grafi . . . 33
4.2 Rappresentazione di un grafo . . . 34
4.3 Visita in Ampiezza di un grafo BFS . . . 35
4.3.1 PseudoCodice BFS . . . 35
4.3.2 Codice C BFS . . . 36
4.3.3 Proprietà delle distanze . . . 38
4.3.4 Proprietà del limite superiore . . . 38
4.3.5 Proprietà della coda . . . 39
4.3.6 Correttezza della BFS . . . 39
4.4 Visita in profondità DFS . . . 40
4.4.1 Pseudo Codice DFS . . . 40
4.4.2 Codice C DFS . . . 41
4.4.3 Teorema delle parentesi . . . 42
4.4.4 Teorema del cammino bianco . . . 42
4.4.5 Tipi di archi DFS . . . 43
4.4.6 Ordinamento Topologico . . . 44
4.4.7 Componenti fortemente connesse . . . 44
4.5 Alberi di connessione minimi MST . . . 44
4.5.1 Il problema degli alberi di connessione minimi . . . 44
4.5.2 Definizioni preliminari . . . 44
4.5.3 Algoritmo greedy per generare un albero di connessione minimo . . . 45
4.5.4 Teorema: Caratterizzazione degli archi sicuri . . . 45
4.5.5 Algoritmo di Kruskal . . . 47
4.5.6 L'algoritmo di Prim . . . 49
4.6 Cammini minimi da sorgente unica . . . 51
4.6.1 Archi di peso negativi . . . 52
4.6.2 Proprietà dei cammini minimi . . . 52
4.6.3 Tecnica del rilassamento . . . 53
4.6.4 Algoritmo di Bellman-Ford . . . 53
4.6.5 Algoritmo di Dijkstra . . . 54

5 Gli alberi **56**
5.1 Tipi di albero . . . 56
5.2 Gli alberi binari . . . 56
5.2.1 Rappresentazione degli alberi binari . . . 56
5.2.2 Albero binario di ricerca BST . . . 57
5.2.3 Visita di un albero binario . . . 57
5.2.4 Analisi della complessità . . . 57
5.2.5 Interrogazioni di un albero binario di ricerca . . . 58
5.3 Alberi Rosso-Neri . . . 62
5.3.1 Proprietà di un albero rosso-nero . . . 62

6 Hashing **63**

6.1 Tabelle ad indirizzamento diretto . . . 63
6.1.1 Operazioni su una tabella ad indirizzamento diretto . . . 63
6.1.2 Fattore di carico . . . 64
6.2 Tavole o Tabelle di hash . . . 64
6.2.1 Metodo del concatenamento (Liste di trabocco) . . . 65
6.2.2 Operazioni sulle liste di trabocco . . . 65
6.2.3 Fattore di carico . . . 65
6.2.4 Teoremi metodo del concatenamento . . . 66
6.2.5 Funzioni di hash . . . 68
6.2.6 Hashing statico . . . 68
6.2.7 Hashing randomizzato . . . 69
6.2.8 Progettare una famiglia universale di funzioni hash – Metodo basato sulla teoria dei numeri . . . 69
6.3 Metodo dell'indirizzamento aperto . . . 70
6.3.1 Ispezione lineare . . . 71
6.3.2 Ispezione quadratica . . . 71
6.3.3 Ispezione con doppio hashing . . . 71

7 Altri Algoritmi **72**
7.1 Problema della Selezione . . . 72
7.1.1 PseudoCodice Min_Max . . . 72
7.2 Selezione in tempo atteso lineare . . . 74
7.2.1 PseudoCodice . . . 74
7.2.2 Analisi . . . 75
7.3 Problema delle inversioni . . . 75

8 Altro **76**
8.1 Modello di Calcolo . . . 76

1 Algoritmi

Informalmente, un **algoritmo** è una procedura di calcolo che prende un certo valore, o un insieme di valori, come input e genera un valore, o un insieme di valori, come output. Un algoritmo è quindi una sequenza di passi computazionali che trasforma l'input in output. Possiamo anche considerare un algoritmo come uno strumento per risolvere un **problema computazionale**

1.1 Il problema dell'ordinamento

Supponiamo di dovere ordinare una sequenza di numeri in ordine non decrescente. Il **problema dell'ordinamento** può essere formalmente definito nel seguente modo:

Input: una sequenza di n numeri $\{a_1, a_2, ..., a_n\}$
Output: una permutazione $\{a'_1, a'_2, ..., a'_n\}$ della sequenza in input, tale che $a'_1 \leq a'_2 \leq ... \leq a'_n$

Per esempio, data la sequenza di input $\{31, 41, 59, 26, 41, 58\}$, un algoritmo di ordinamento restituisce come output la sequenza $\{26, 31, 41, 41, 58, 59\}$. Tale sequenza di input è detta **istanza** del problema dell'ordinamento. In generale, l'istanza di un problema è formata dall'input (che soddisfa tutti i vincoli imposti nella definizione del problema) richiesto per calcolare una soluzione del problema.

L'**ordinamento** è un'operazione fondamentale in informatica, per questo abbiamo a disposizione molti algoritmi di ordinamento. La scelta dell'algoritmo più appropriato a una data applicazione dipende dal numero di elementi da ordinare, dal livello di ordinamento iniziale degli elementi, da eventuali vincoli sui valori degli elementi, dall'architettura del calcolatore e dal tipo di unità di memorizzazione da utilizzare.

Un algoritmo si dice **corretto** se, per ogni istanza di input, termina con l'output corretto. Diciamo che un algoritmo corretto risolve il problema computazionale dato (in cui l'istanza rappresenta un problema appartenente ad una specifica classe).

1.2 Costo di un algoritmo

1.2.1 Definizione

Un algoritmo A ha costo O(g(n)) se la quantità di tempo (spazio) sufficiente per eseguire A su un input di dimensione n nel caso peggiore è O(g(n)).

Ricorda: i costi in generale possono essere al più polinomiali; infatti, se ad esempio i costi di un algoritmo nel caso peggiore fossero esponenziali si avrebbe che per $n \to \infty$ tenderebbero ad infinito e questo a livello fisico è impossibile.

Espresso sotto forma di funzione **f(n)** significa che n rappresenta la dimensione dell'input. Tuttavia per non confondere tra **f(int n)** e f(n), spesso è indicato con f(|x|), dove x è l'input e |x| è la dimensione effettiva dell'input. Quindi, se stessimo considerando un algoritmo che opera su una stringa x, ad esempio, potremmo indicare il costo dell'algoritmo come **f(|x|)**, dove |x| rappresenta la lunghezza della stringa.

Stabilire il costo computazionale di un algoritmo serve a confrontare diversi algoritmi che risolvono lo stesso problema, in modo da scegliere il più efficiente. Una misura del costo per essere soddisfacente deve:

- basarsi su un modello di calcolo astratto, ovvero deve essere indipendente dalla particolare macchina;

- svincolarsi dalla configurazione dei dati in ingresso, ad esempio basandosi sul caso peggiore;
- essere una funzione della dimensione dell'input;
- essere asintotica, ovvero fornire un'idea dell'andamento del costo all'aumentare dell'input.

Il costo di un algoritmo si riferisce alle risorse necessarie per l'esecuzione dell'algoritmo stesso. Queste risorse possono includere il **tempo di elaborazione** (complessità temporale) e lo **spazio di memoria** (complessità spaziale).

1.2.2 Complessità temporale

La **complessità temporale** di un algoritmo è una misura del tempo di esecuzione di un algoritmo in funzione della dimensione dei dati in input. Questa complessità cresce al crescere della dimensione dei dati in input.

1.2.3 Complessità spaziale

La **complessità spaziale** di un algoritmo è una misura delle risorse hardware utilizzate durante l'esecuzione di un algoritmo. Se un algoritmo richiede più memoria per elaborare un insieme di dati più grande, allora ha una complessità spaziale più alta.

1.2.4 Istruzione dominante

Quando si analizza un algoritmo in cui sono presenti molti confronti, assegnazioni ecc..., è complicato calcolare il costo, in tal caso si prende in considerazione l'istruzione dominante, ovvero quell'operazione che si ripete più volte nel programma, tipicamente quella contenuta nei **cicli più interni**.

1.2.5 Regola di Horner

La regola di Horner, o più correttamente, l'algoritmo di Horner, è un metodo efficiente per valutare un polinomio e ridurre il costo computazionale da quadratico a lineare. Supponiamo di avere un polinomio P(x) di grado n: $P(x) = a_n x^n + a_{n-1}x^{n-1} + ... + a_1 x + a_0$. Ciò che fa l'algoritmo è ridurre il polinomio P(x) ad una serie di operazioni di moltiplicazione e addizione annidate: $P(x) = \sum_{i=0}^{n} a_i x^i = x \sum_{i=0}^{n} a_i x^{i-1}$, così un generico polinomio in x è uguale $a_0 + x \cdot (polinomio\ di\ grado\ inferiore)$. Il vantaggio di questo metodo è che richiede solo n moltiplicazioni e n addizioni, indipendentemente dal grado del polinomio. Questo è significativamente più efficiente rispetto al metodo standard di valutazione di un polinomio, che richiede n^2 moltiplicazioni e n addizioni.

1.3 Andamento asintotico

L'andamento asintotico di un algoritmo è una misura della crescita del tempo di esecuzione o dello spazio richiesto quando le dimensioni dell'input crescono. In altre parole, l'andamento asintotico ci dice come si comporta l'algoritmo quando l'input diventa grande, ovvero quando cresce la sua dimensione, il numero di bit di cui si ha bisogno per rappresentarlo (se l'input è pari ad n, la sua dimensione sarà pari a $\lceil \log_2(n+1) \rceil$). Il logaritmo in base 2 di un numero rappresenta esattamente il numero di bit necessari per rappresentarlo. Questo è dovuto al fatto che ogni bit in una rappresentazione binaria può essere o 0 o 1, quindi c'è una corrispondenza diretta tra i bit necessari per rappresentare un numero e il logaritmo in base 2 di quel numero.

L'andamento asintotico è utile perché ci consente di fare previsioni sulle prestazioni di un algoritmo su input grandi senza dover eseguire l'algoritmo su input effettivamente grandi. Ci sono due notazioni principali utilizzate per esprimere l'andamento asintotico di un algoritmo:

- **Notazione O grande (Big O).** Questa notazione fornisce un limite superiore per la crescita del tempo di esecuzione o dello spazio utilizzato dall'algoritmo, nel caso peggiore. Ad esempio, se un algoritmo ha un andamento asintotico di O(n), significa che il tempo di esecuzione o lo spazio utilizzato cresce al massimo proporzionalmente al numero di elementi nell'input (n).

- **Notazione Ω grande (Omega).** Questa notazione fornisce un limite inferiore per la crescita del tempo di esecuzione o dello spazio utilizzato dall'algoritmo, nel caso migliore. Ad esempio, se un algoritmo ha un andamento asintotico di Ω(n), significa che il tempo di esecuzione o lo spazio utilizzato cresce almeno proporzionalmente al numero di elementi nell'input (n).

Spesso, si utilizza anche la notazione **Θ(Theta)**, che indica un limite stretto superiore e inferiore per la crescita dell'algoritmo. Ad esempio, se un algoritmo ha un andamento asintotico di Θ(n), significa che il tempo di esecuzione o lo spazio utilizzato cresce esattamente proporzionalmente al numero di elementi nell'input (n).

Il **comportamento asintotico** del costo di un programma si basa sull'idea di trascurare costanti moltiplicative e termini di ordine inferiore, fornendo un valore che cambia man mano che la dimensione aumenta. Ad esempio, il costo $\mathbf{an + b}$ ha comportamento asintotico lineare, il costo $\mathbf{an^2 + b}$ ha comportamento asintotico quadratico, il costo $\mathbf{a \cdot \log_b n + c}$ ha comportamento asintotico logaritmico.

Per calcolare la **complessità di un algoritmo** si utilizza l'analisi asintotica che trasforma il tempo di esecuzione dell'algoritmo in una funzione T(n) in funzione della dimensione n dei dati in input, con la quale misurare il numero di comandi eseguiti dall'algoritmo. Ad esempio, data un'istanza di dimensione n, nel caso peggiore l'algoritmo ha complessità temporale pari a O(f(n)) se T(n) = O(f(n)). Dove n è il numero delle righe eseguite mentre f(n) è un limite superiore del tempo di esecuzione.

1.3.1 Casi da ricordare

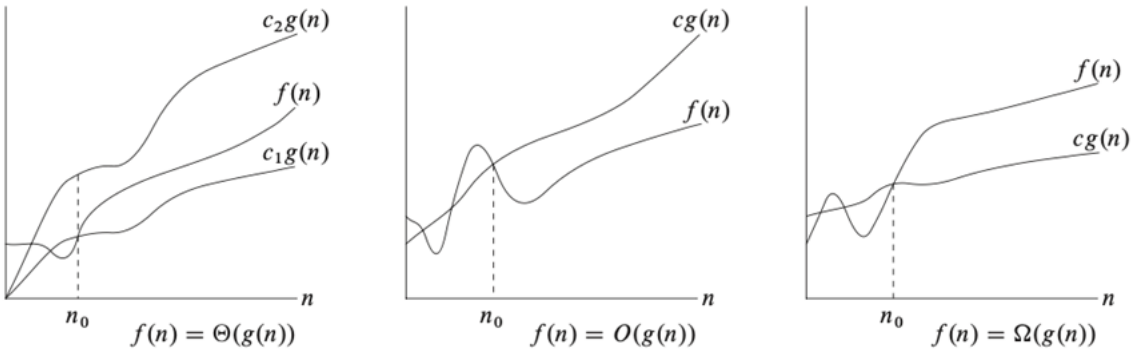
- **O(g(n))**: Si dice che **f(n) è O(g(n))** se esiste un valore $c > 0$ tale che $0 \leq f(n) \leq c \cdot g(n)$ per ogni $n \geq n_0$. Questa notazione rappresenta un limite superiore asintotico, cioè descrive un limite superiore alla crescita di f(n). In altre parole, f(n) cresce al massimo come g(n). Ad esempio, se consideriamo un programma che ha tempo di esecuzione $T(n) = (n+1)^2$, allora T(n) è $O(n^2)$. Infatti se poniamo c = 2 e $n_0 = 3$ abbiamo $(n+1)^2 \leq 2 \cdot n^2$ per ogni $n \geq 3$.

- **Ω(g(n))**: Si dice che **f(n) è Ω(g(n))** se esiste un valore $c > 0$ tale che $0 \leq c \cdot g(n) \leq f(n)$ per ogni $n \geq n_0$. Questa notazione rappresenta un limite inferiore asintotico, cioè descrive un limite inferiore alla crescita di f(n). In altre parole, f(n) cresce almeno come g(n).

- **Θ(g(n))**: Si dice che **f(n) è Θ(g(n))** se esistono due valori $c_1 > 0$ *e* $c_2 > 0$ tali che $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ per ogni $n \geq n_0$. Questa notazione rappresenta un limite asintotico stretto, cioè descrive sia un limite superiore che un limite inferiore alla crescita di f(n). In altre parole, f(n) cresce esattamente come g(n). Dunque è come se avessimo trovato upper e lower bound.

I tre casi possono anche essere interpretati come insiemi e quindi rispettivamente avremo:

- $f(n) \in O(g(n))$
- $f(n) \in \Omega(g(n))$
- $f(n) \in \Theta(g(n))$ ovvero $f(n) \in O(g(n)) \wedge f(n) \in \Omega(g(n)$



Quando si usa ω (omega minuscolo) si vuole enfatizzare che il **lower bound non è tight**, ottenendo $f(n) \in o(g(n))$ che equivale a scrivere $\lim_{n\to\infty} \frac{f(n)}{g(n)}$. Ricorda che quando si parla di lower bound (limite inferiore) di un problema computazionale o di un algoritmo, dire che è "tight" significa che corrisponde al limite inferiore effettivo o che è vicino a esso. In altre parole, il lower bound è tight quando si dimostra che nessun algoritmo può risolvere il problema in un tempo migliore di quanto indicato dal lower bound stesso. Ad esempio, se dimostriamo che il lower bound per un problema di ordinamento è $\Omega(n \cdot logn)$, significa che nessun algoritmo di ordinamento può fare meglio di $\Omega(n \cdot logn)$ nel caso peggiore. Se un particolare algoritmo di ordinamento raggiunge questo lower bound, allora si dice che il lower bound è tight per quel particolare algoritmo.

1.3.2 Problemi

Esistono diversi tipologie di problemi: decisionali (una domanda la cui risposta è sì o no), di calcolo e di ottimizzazione.

Definizione: Il problema P ha ub (upper bound) pari a f(n) se e solo se $\exists A \mid (A\ risolve\ P) \wedge A \in O(f(n))$. Il che si traduce nel dire "se e solo se esiste un algoritmo tale per cui esso risolve il problema P ed appartiene all'insieme O(f(n))".

Ad esempio, il problema Q ammette due algoritmi risolutivi se $A \in O(f(n))\ e\ B \in O(g(n))$ e, f(n) o(g(n)) (significa che g(n) è più grande di f(n)).

Definizione: Il problema P ha lb (lower bound) pari a h(n) se e solo se $\forall A \mid (A\ risolve\ P) \wedge A \in \Omega(h(n))$.

Inoltre ricorda che se l'input ha dimensione n allora $\Omega(n)$ lb banale, infatti se ho dimensione di input n, il costo è lineare.

Ecco un esempio per capire meglio i concetti con un'equazione di secondo grado: $ax^2+bx+c = 0$. Possiamo scegliere a = b = c = 1 e ottenere $x^2 + x + 1 = 0$, oppure scegliere a = 1, b = 2, c = 3 e ottenere $x^2 + 2x + 3 = 0$. Risolvere le due equazioni significa risolvere un problema che appartiene alla stessa classe, ovvero categoria di problemi, dunque la prima e la seconda equazione sono "simili". Non avendo input è difficile definire sia ub che lb. O(1) : ub è la funzione costante $\Omega(1)$: lb è la funzione costante $\Theta(1)$

Definizione: Un problema può essere definito da una classe perché, in informatica, una classe è una struttura che definisce le proprietà e i comportamenti comuni a un gruppo di oggetti. Un'istanza è un esemplare specifico di una classe, in quanto tale ha le stesse proprietà della classe, ma i valori di queste proprietà possono variare da istanza a istanza.

2 Tecniche di progettazione degli algoritmi

Esistono tecniche comuni, come il metodo divide et impera, la randomizzazione e la soluzione delle ricorrenze, per la progettazione e analisi di algoritmi efficienti. A queste si aggiungono tre importanti tecniche più sofisticate e mirate per specifiche categorie di problemi computazionali tra cui la programmazione dinamica e gli algoritmi golosi.

2.1 La programmazione dinamica

La programmazione dinamica è una tecnica algoritmica che viene applicata per risolvere problemi di ottimizzazione nei quali si arriva a una soluzione ottima effettuando una serie di scelte. È efficace quando il problema principale può essere scomposto in sottoproblemi che non sono indipendenti e possono ricomparire più volte. Il concetto chiave consiste nel memorizzare le soluzioni dei sottoproblemi per poterle usare nel caso questi si ripresentassero. La soluzione ottimale del problema può essere ottenuta combinando soluzioni ottimali di sottoproblemi. Questo concetto, a volte, consente di trasformare algoritmi con tempi esponenziali in algoritmi con tempi polinomiali.

2.1.1 Problema delle aste

Un classico esempio è il problema del taglio delle aste. Si considera un'asta metallica di lunghezza n che deve essere tagliata in pezzi di lunghezza intera per massimizzare il ricavo della vendita dei pezzi ottenuti. Per ogni lunghezza l da 1 a n è dato un prezzo p_l a cui si possono vendere i pezzi di quella lunghezza. L'obiettivo è decidere come tagliare l'asta in modo da rendere massimo il ricavo della vendita.

Esempio di calcolo del ricavo
Consideriamo un'asta di lunghezza 4 con i seguenti prezzi per le varie lunghezze.

Lunghezza l	$\mathbf{p_l}$
1	1
2	5
3	8
4	9

Si può costruire una tabella per calcolare il ricavo massimo possibile per ogni lunghezza dell'asta.

Lunghezza asta n	**Ricavo massimo $\mathbf{r_n}$**	**Suddivisione ottima**
1	1	1
2	5	2
3	8	3
4	10	2+2

Un'asta di lunghezza n può essere tagliata in 2^{n-1} modi distinti in quanto abbiamo la possibilità di scegliere se tagliare o meno in ogni posizione intera 1, . . . , n-1. Ad esempio per n = 4

esistono i seguenti possibili modi.

4	1 + 1 + 2
1 + 3	1 + 2 + 1
2 + 2	2 + 1 + 1
3 + 1	1 + 1 + 1 + 1

Formulazione ricorsiva
La soluzione ottimale del problema del taglio delle aste può essere formulata in modo ricorsivo. Il ricavo massimo r_n per un'asta di lunghezza n è il massimo tra:

- Il prezzo dell'asta intera p_n
- La somma del ricavo massimo dei pezzi ottenuti tagliando l'asta in due parti di lunghezza i e n-i, dove i varia da 1 a n-1 ($r_n = r_i + r_{n-i}$);

$$r_n = \begin{cases} p_1 & \text{se n} = 1 \\ max(p_n,\ r_1 + r_{n-1},\ r_2 + r_{n-2},\ ...,\ r_{n-1} + r_1) & \text{se n} > 1 \end{cases}$$

Anziché tagliare l'asta per la prima volta in posizione i, è possibile scegliere la lunghezza i del primo pezzo per cui $r_n = p_i + r_{n-i}$.

$$r_n = \begin{cases} 0 & \text{se n} = 0 \\ max_{\ 1 \le i \le n}\ (p_i + r_{n-1}) & \text{se n} > 0 \end{cases}$$

In questo modo si registra una perdita di efficienza nell'algoritmo, dovuta al fatto che vi sarebbero dei sottoproblemi ripetuti, per poter evitare ciò si possono usare due approcci:

- **Metodo Top-Down con Memorizzazione:** memorizzare i risultati dei sottoproblemi già risolti;
- **Metodo Bottom-up:** calcolare prima i problemi più piccoli memorizzandone le soluzioni e poi usare tali soluzioni per risolvere i problemi più grandi.

Con questi due approcci si otterrebbe la seguente equazione di ricorrenza $\mathbf{T(n) = \Theta(n^2)}$.

```
CUT-ROD(p, n)
 if n == 0
  return 0
 q= INT_MIN
 for i = 1 to n
  q = max(q, p[i] + CUT-ROD(p, n-i)
 return q
```

2.1.2 Moltiplicazione di Matrici

Un altro problema risolvibile con la programmazione dinamica è la moltiplicazione di matrici. Supponiamo di avere una catena di matrici $A_1,\ A_2,\ \ldots,\ A_n$ e vogliamo trovare il modo ottimale per moltiplicarle minimizzando il numero totale di moltiplicazioni tra scalari. Poiché la moltiplicazione di matrici è associativa, possiamo calcolare il prodotto delle matrici in molti modi diversi. Ogni modo corrisponde a una diversa parentesizzazione dell'espressione.
Supponiamo di avere tre matrici A_1 $(10x20)$, A_2 $(20x30)$ e A_3 $(30x40)$. Le due parentesizzazioni possibili sono

$$(A_1 \cdot A_2) \cdot A_3$$

$$A_1 \cdot (A_2 \cdot A_3)$$

Il numero di parentesizzazioni possibili $P(n)$ del prodotto di n matrici si esprime ricorsivamente come segue:

$$P(n) = \begin{cases} 1 & \text{se n} = 1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & \text{se n} > 1 \end{cases}$$

$P(n)$ cresce in modo esponenziale. Per risolvere il problema è innanzitutto necessario determinare il numero di moltiplicazioni scalari necessarie per i prodotti tra le matrici in ogni parentesizzazione e, scegliere la parentesizzazione che richiedere il numero minimo di moltiplicazioni.

1. **Caratterizzare la struttura di una soluzione ottima:** una soluzione al problema della parentesizzazione ottima di n matrici divide il problema nei due sottoproblemi seguenti, ovvero il prodotto delle prime k matrici e il prodotto delle rimanenti n-k. La soluzione finale $A_{1\ldots n}$ è il prodotto delle due matrici $A_{1\ldots k}$ e $A_{k+1\ \ldots\ n}$;

2. **Definire ricorsivamente il valore di una soluzione ottima:** ciò viene fatto sulla base del costo minimo per il prodotto di matrici. Per calcolare il prodotto $A_{i\ldots j}$, il costo per il caso base vale 0, ovvero quando i = j, mentre ricorsivamente, il caso induttivo porta il seguente risultato per $i < j$: $C(i,j) = min\ _{i\leq\ k\ \leq j}\ (C(i,k) + C(k+1,j) + p_{i-1}p_kp_j)$ dove p indica la dimensione delle matrici;

3. **Calcolare il valore di una soluzione ottima**;

4. **Costruzione di una soluzione ottima**.

$$C(i,j) = \begin{cases} 0 & se\ i = j \\ min\ _{i\leq\ k\ \leq j}\ (C(i,k) + C(k+1,j) + p_{i-1}p_kp_j) & se\ i < j \end{cases}$$

2.1.3 Conclusione

La programmazione dinamica è applicabile a problemi definiti di ottimizzazione in cui da un insieme, generalmente molto grande, di soluzioni possibili si vuole estrarre una soluzione ottima rispetto ad una determinata misura. Affinché sia vantaggiosa l'applicazione di questa tecnica è necessario vengano soddisfatte le seguenti proprietà

1. **Proprietà di struttura ottima:** un problema di ottimizzazione ha una struttura ottima se una soluzione ottimale contiene soluzioni ottimali ai suoi sottoproblemi;

2. **Proprietà della ripetizione dei sottoproblemi:** è necessario che il numero di sottoproblemi distinti sia minore del numero di soluzioni possibili tra cui cercare quella ottima. Altrimenti una enumerazione di tutte le soluzioni può risultare più conveniente.

Se le due proprietà sono verificate occorre scegliere l'ordine in cui calcolare le soluzioni dei sottoproblemi

1. **Algoritmo iterativo bottom-up:** migliore se per il calcolo della soluzione globale servono le soluzioni di tutti i sottoproblemi. L'ordine trovato con questo algoritmo garantisce che risolvendo un sottoproblema, le soluzioni dei sottoproblemi da cui esso dipende siano già state calcolate;

2. **Algoritmo ricorsivo top-down con memoizzazione:** migliore se per il calcolo della soluzione globale servono soltanto alcune delle soluzioni dei sottoproblemi. Questo algoritmo esprime direttamente la soluzione di un sottoproblema in termini delle soluzioni dei sottoproblemi da cui essa dipende. In questo caso è tuttavia necessario memorizzare le soluzioni trovate affinchè non vengano più ricalcolate.

2.2 Algoritmi golosi

Per molti problemi di ottimizzazione è uno spreco applicare le tecniche della programmazione dinamica per effettuare le scelte migliori, si preferisce ad esempio un algoritmo goloso (greedy). Questo fa sempre la scelta che sembra ottima in un determinato momento, ovvero fa una scelta localmente ottima, nella speranza che tale scelta porterà a una soluzione globalmente ottima. Il metodo goloso ha varie applicazioni che lo rispecchiano, come gli algoritmi per gli alberi di connessione minimi, l'algoritmo di Dijkstra per il problema dei cammini minimi da sorgente unica.

2.2.1 Problema della selezione di attività

Supponiamo di avere un insieme $S = a_1,\ a_2,\ ...,\ a_n$ di n attività che devono utilizzare la stessa risorsa, per esempio un'aula universitaria, che può essere utilizzata per svolgere una sola attività alla volta. Dunque l'obiettivo è quello di selezionare il più grande insieme di attività mutualmente compatibili, cioè che non si sovrappongano nei tempi di inizio e fine.
Ogni attività a_i ha un tempo di inizio s_i e un tempo di fine f_i, con $s_i < f_i$. Quando viene selezionata, l'attività a_i si svolge durante l'intervallo $[s_i,\ f_i)$. Le attività a_i e a_j sono compatibili se i seguenti intervalli sono disgiunti $[s_i,\ f_i)$ e $[s_j,\ f_j)$. Affinché sia possibile implementare la giusta strategia è necessario supporre che le n attività siano ordinate per tempo di fine non decrescente $f_1 \leq ... \leq f_n$, altrimenti andrebbero ordinate in questo modo in un tempo con costo pari a $O(n\ log\ n)$.

Strategie non funzionanti: scegliere l'attività che inizia prima, scegliere l'attività che dura meno, scegliere l'attività incompatibile con il minor numero di altre attività.

Strategia funzionante: tra le attività che si possono scegliere, ce ne deve essere una che finisce per prima, scegliendo questa la risorsa resterebbe disponibile per il maggior numero possibile di attività successive. Se più attività in S hanno il primo tempo di fine, si sceglie una qualsiasi tra queste. Dopodiché è necessario trovare le attività che iniziano dopo la fine di questa, in questo modo sarebbero compatibili.
L'algoritmo sceglie la prima attività a_1, ovvero quella con tempo di fine minimo, esiste sempre una soluzione ottima che contiene a_1? La risposta è sì, sia $b_1,\ ...,\ b_j$ una qualsiasi soluzione ottima che supponiamo ordinata per tempo di fine. Le attività $b_2,\ ...,\ b_m$ sono compatibili con b_1 e terminano dopo b_1. Esse hanno quindi tempo di inizio maggiore o uguale al tempo di fine f di b_1. Ma a_1 ha tempo di fine f_1 minimo in assoluto quindi termina prima di b_1. Quindi anche a_1 è compatibile con $b_2,\ ...,\ b_m$. Dunque $a_1,\ b_2,\ ...,\ b_m$ è una soluzione ottima che contiene a_1.

```
Greedy-Activity-Selector(s, f, n)
    A = {a_1}, k = 1
    for m = 2 to n
        if s[m] >= f[k]
            A = A union {a_m}
            k = m
    return A
```

Note sullo pseudocodice

I tempi di inizio e fine sono dati nei due array s e f. Inoltre k viene posto ad 1 ed aggiornato ad m ogni volta che si sceglie una nuova attività a_m. Essendo le attività ordinate per tempo di fine non decrescente, $f[k]$ è il massimo tempo finale delle attività selezionate precedentemente. Con il test dell'if l'algoritmo seleziona la prima attività a_m il cui tempo di inizio $s[m]$ è maggiore o uguale di $f[k]$.
Esiste sempre una soluzione ottima che contiene a_m e le attività finora scelte? La risposta è sì, sia $b_1, \ldots, b_i, b_{i+1}, \ldots, b_j$ che estende le attività $b_1, \ldots, b_i$ finora scelte e supponiamo $b_1, \ldots, b_i$ e $b_{i+1}, \ldots, b_j$ ordinate per tempo di fine. Le attività scartate finora iniziano prima e hanno tempo di fine maggiore o uguale ad una delle attività precedentemente scelte. Esse sono quindi incompatibili con almeno una delle attività $b_1, \ldots, b_k$ finora scelte. $b_{k+1}, \ldots, b_m$ sono invece compatibili con tutte le attività $b_1, \ldots, b_k$ e quindi non sono tra quelle scartate precedentemente. $b_{k+1}, \ldots, b_m$ hanno sia tempo di fine che tempo di inizio maggiore o uguale di $f[k]$. L'attività a_i è quella con tempo di fine f_i minimo in assoluto tra quelle compatibili con $b_1, \ldots, b_k$ e quindi $f_i \leq f$. Siccome b_{k+1} è compatibile con $b_{k+2}, \ldots, b_m$ i tempi di inizio di $b_{k+2}, \ldots, b_m$ sono maggiori o uguali di f e quindi anche di f_i. Dunque anche a_i è compatibile con $b_{k+2}, \ldots, b_m$. Pertanto $b_1, \ldots, b_k, a_1, b_{k+2}, \ldots, b_m$ è una soluzione ottima contenente a_i e $b_1, \ldots, b_k$.
Si può concludere dicendo che durante tutta l'esecuzione dell'algoritmo esiste sempre una soluzione ottima contenente le attività $b_1, \ldots, b_i$ scelte fino a quel momento. Quando l'algoritmo termina non ci sono altre attività compatibili con $b_1, \ldots, b_i$ e quindi le attività $b_1, \ldots, b_i$ sono una soluzione ottima.

2.2.2 Procedimento per sviluppare un algoritmo avido o goloso

1. Esprimere il problema di ottimizzazione in una forma in cui, fatta una scelta, resta un solo sottoproblema da risolvere;

2. Dimostrare che esiste sempre una soluzione ottima del problema originale che fa la scelta golosa, quindi la scelta golosa è sempre sicura;

3. Dimostrare la sottostruttura ottima verificando che, dopo aver fatto la scelta golosa, ciò che resta è un sottoproblema con la proprietà che, se combiniamo una soluzione ottima nel sottoproblema con la scelta golosa che abbiamo fatto, arriviamo a una soluzione ottima del problema originale.

2.2.3 Elementi della strategia avida

Sottostruttura ottima: Un problema di ottimizzazione ha una sottostruttura ottima se una soluzione ottima del problema contiene al suo interno soluzioni ottime dei sottoproblemi.

Proprietà della scelta avida: La scelta ottima localmente, ovvero quella golosa, non impedisce di trovare una soluzione globalmente ottima.

2.2.4 Algoritmo di Huffman

L'algoritmo di Huffman è una procedura per costruire un albero di codifica ottimale per un insieme di simboli con frequenze associate, garantendo che i simboli più frequenti abbiano codici più corti e i simboli meno frequenti abbiano codici più lunghi.

Strategia dell'algoritmo

1. **Creazione dei nodi:** Ogni simbolo e la sua frequenza vengono considerati come nodi di un albero. Ogni nodo è inizialmente un albero singolo.
2. **Costruzione dell'albero:**
   - Si selezionano i due nodi con le frequenze più basse.
   - Si crea un nuovo nodo che ha questi due nodi come figli e la cui frequenza è la somma delle frequenze dei due nodi.
   - Si inserisce il nuovo nodo nella lista dei nodi.
   - Si ripete il processo fino a quando c'è un solo nodo nella lista. Questo nodo diventa la radice dell'albero di Huffman.
3. **Assegnazione dei codici:** Si assegnano i codici ai simboli seguendo i cammini nell'albero dall'omonima radice alle foglie. Convenzionalmente, si assegna '0' per i rami di sinistra e '1' per i rami di destra.

Pseudo Codice

```
Huffman(C){
    n = |C|, Q = C
    // Q coda di min-priorita'
    for i = 1 to n-1 // costruita in tempo O(n)
        crea nuovo nodo z // con Build-Min-Heap
        x = Extract-Min(Q)
        y = Extract-Min(Q)
        z.left = x
        z.right = y
        z.freq = x.freq + y.freq
        Insert(Q, z))

    return Extract-Min(Q) // alla fine resta solo la radice
}
```

3 Algoritmi di ordinamento (sorting)

Ordinamento stabile (Stable Sorting)

Un algoritmo di ordinamento si dice stabile se preserva l'ordinamento dei relativi dati. Ad esempio, se si ordina per anno di corso una lista di studenti già ordinata alfabeticamente, un metodo stabile produce una lista in cui gli alunni dello stesso anno sono ancora in ordine alfabetico mentre un ordinamento instabile probabilmente produrrà una lista senza più alcuna traccia del precedente ordinamento. Tuttavia bisogna fare attenzione agli elementi che sono uguali, infatti se A = B, affinché l'algoritmo sia stabile è necessario poter stabilire che A e B siano nella stessa posizione.

Algoritmi IN-PLACE

Alcuni algoritmi di ordinamento possono ordinare una lista senza crearne una nuova. Questi sono conosciuti come algoritmi di ordinamento in loco (o in place) e richiedono uno spazio di memoria extra costante O(1). InsertionSort e QuickSort sono **IN-PLACE**, in quanto gli elementi vengono spostati internamente all'array e non allocano memoria aggiuntiva. MergeSort **non è IN-PLACE**, richiede memoria aggiuntiva per i diversi frammenti generati.

Definizione 1. Un algoritmo si definisce IN-PLACE quando è in grado di restituire l'output utilizzando soltanto un piccolo e costante spazio di memoria extra, i dati in ingresso sono solitamente sovrascritti.
Definizione 2. Un algoritmo si dice IN-PLACE se al crescere dell'input non crescono le variabili di appoggio.

3.1 Insertion Sort

Il funzionamento dell'insertion sort è il seguente: immaginiamo l'array come se fosse diviso in due parti: una ordinata e l'altra no, ricordando che un array di dimensione unitaria è banalmente ordinato. Successivamente l'algoritmo itera sulla porzione non ordinata dell'array prendendo un elemento alla volta e confrontandolo con gli elementi dell'array già ordinati. Così facendo viene trovata la nuova posizione dell'elemento preso in considerazione nel sotto-array ordinato.

3.1.1 Codice C iterativo InsertionSort

```
void InsertionSort(int arr [], int n) {
  int i, key, j;
  for (i = 1; i < n; i++) {
    key = arr[i];
    j = i - 1;
    while (j >= 0 && arr[j] > key) {
      arr[j + 1] = arr[j];
      j = j - 1;
    }
    arr[j + 1] = key;
  }
}
```

3.1.2 Esempio iterativo InsertionSort

Consideriamo un array di sei elementi A= [8 3 5 1 4 2]

A= [8 **3** 5 1 4 2]
Step 1: si parte dall'elemento in posizione 1 che sarà la key da confrontare con gli elementi precedenti. In questo caso, key viene confrontato con 8, dato che 8 > 3, 8 e il 3 vengono scambiati. [**3 8** 5 1 4 2]

A= [3 8 **5** 1 4 2]
Step 2: si parte dall'elemento in posizione 2 che sarà la nuova key, 8 > 5 allora: [3 **8 8** 1 4 2] e successivamente viene posizionato key nella posizione corretta: [3 **5 8** 1 4 2]

A= [3 5 8 **1** 4 2]
Step 3: si parte dall'elemento in posizione 3 che sarà la nuova key, 8 > 1 allora: [3 5 **8 8** 4 2], 5 > 1 allora [3 **5 5** 8 4 2], 3 > 1 allora [**3 3** 5 8 4 2] ed infine [**1 3 5 8** 4 2].

A= [1 3 5 8 **4** 2]
Step 4: si parte dall'elemento in posizione 4 che sarà la nuova key, 8 > 4 allora [1 3 5 **8 8** 2], 5 > 4, 3 > 4? No, allora ci si ferma.

A= [1 3 4 5 8 **2**]
Step 5: si parte dall'elemento in posizione 5 che sarà la nuova key, 8 > 2, 5 > 2, 4 > 2, 3 > 2, 1 > 2? No, allora ci si ferma e si ottiene [1 2 3 4 5 8].

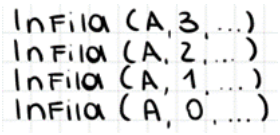
3.1.3 Codice C ricorsivo InsertionSort

```
void infila(int A[], int i, int ultimo){ // i = primo
    int h = i+1;
    int k = A[i];
    while((A[h] < k) && (h<=ultimo){
        A[h−1]= A[h];
        h++;
    }
    A[h−1]= k;
    return;
}
int ordina(int A[], int primo, int ultimo){
    if(ultimo > primo)
        infila (A[], primo, ordina(primo + 1, ultimo);
    return ultimo;
}
```

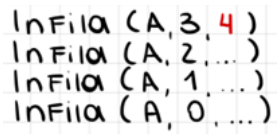
3.1.4 Esempio ricorsivo InsertionSort

Consideriamo un array di cinque elementi A= [5 3 6 4 1].
Viene eseguita la prima volta la funzione *ordina* alla quale si passano il primo e l'ultimo indice dell'array, ovvero le posizioni del primo e dell'ultimo elemento. Eseguendo l'if per la prima volta si controlla se 4 > 0 allora si esegue la chiamata ad *infila* lasciata in sospeso perché contenente la chiamata ricorsiva ad *ordina*, ciò vuol dire che il campo contenente *ultimo* si lascia vuoto. Si richiama *ordina* avendo incrementato *primo* che ora vale 1, dopo aver effettuato il controllo dell'if si procede come prima incrementando *primo*, il che si ripete fino ad arrivare ad avere

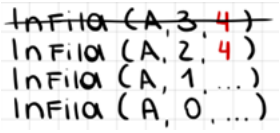
primo = 3:



A questo punto avendo richiamato ricorsivamente *ordina* con *primo = 4* la condizione dell'if non verrà soddisfatta in quanto $4 > 4$ è false e restituendo *ultimo= 4* lo si inserirà nell'ultima chiamata ad *infila* lasciata in sospeso:



Ora si esegue la chiamata a funzione *infila* e si ordina con essa la porzione di array di indici [3, 4] come segue: se A= [5 3 6 4 1] l'elemento in posizione tre è 4 e quello in posizione quattro è 1. Si salva k= 4 e h= i + 1= 3 + 1 essendo *i* la posizione di 4. La condizione del ciclo while sarà rispettata finché l'elemento dell'array in posizione h, in questo caso avendo h= 4 avremo A[4]= 1, sarà minore di k ed $h \leq 4$. Allora l'elemento in posizione h-1= i sarà A[h], ovvero abbiamo messo nel sotto-array l'elemento in ultima posizione nella posizione corretta. Avendo incrementato h, che ora sarà pari a 5, la condizione del ciclo non è rispettata e in posizione h-1= 4 si porrà k. Avremo così ottenuto A= [5 3 6 **1 4**]. Si torna in *ordina* e dopo aver eseguito *infila* si restituisce *ultimo*, ovvero si inserisce nell'altra chiamata ad *infila* lasciata in sospeso.



Eseguendo ora *infila* si ordinerà il sotto-array dell'intervallo di indici [2, 4]: A= [5 3 **6 1 4**]. L'elemento in posizione due è 6, quello in posizione tre è 1, infine quello in posizione quattro è 4. Si salva k= 6 e h= i + 1= 2 + 1= 3 essendo i la posizione di 6. La condizione del ciclo while sarà rispettata finché l'elemento dell'array h, in questo caso avendo h= 3 avremo A[3]= 1, sarà minore di k ed $h \leq 4$. Allora l'elemento in posizione h-1= i sarà A[h] e incrementiamo h, ovvero abbiamo messo nel sotto-array l'elemento in ultima posizione nella posizione corretta. Ora avremo A= [5 3 **1 1 4**], avendo incrementato h ora sarà pari a 4, così da avere A[h]= A[4]= 4 e poter come prima scambiare gli elementi in modo da porre A[h-1]= 4 e, incrementato h a 5 non sarà rispettata la condizione del ciclo while e verrà infine posto A[h-1]= A[4]= k= 6. Si procede in questo modo fino ad ordinare tutto l'array.

3.1.5 InsertionSort: Analisi dei costi

Analizzando il codice C iterativo, si può notare che i cicli risultano essere le operazioni più frequenti. Dunque il ciclo for viene eseguito n-1 volte, mentre il ciclo while, j volte, tuttavia osservando che il valore j viene inizializzato fuori dal while a i-1, si capisce che l'ultimo valore sarà proprio j= (n-1)-1= n-2. Da ciò è possibile intuire che il costo dell'algoritmo sarà $(n-1) \cdot (n-2)$, ovvero il costo asintotico sarà pari a $\mathbf{O(n^2)}$. Il costo dell'algoritmo può anche essere analizzato ragionando con la tecnica dello srotolamento.

3.1.6 Equazione di ricorrenza

Un'equazione di ricorrenza definisce una funzione matematica che descrive la complessità di un algoritmo esprimendone il costo.

3.1.7 Tecnica dello srotolamento

La tecnica dello srotolamento di un equazione di ricorrenza consiste nel "richiamare" l'equazione un certo numero di volte (ogni volta con un argomento di dimensione più piccola) fin quando non è possibile esprimerla come una somma di termini dipendenti solo da n e dalle condizioni iniziali. Esempio:

$$\begin{cases} T(n) = T(n-1) + 1 & \text{passo ricorsivo} \\ T(1) = 1 & \text{passo base} \end{cases}$$

Si procede adesso srotolando la ricorsione:
$T(n) = T(n-1) + 1$

Si sviluppa il termine $\mathbf{T(n-1)}$ $\rightarrow T(n-1) = T(n-2) + 1$
$T(n) = T(n-2) + 2$

Si sviluppa il termine $\mathbf{T(n-2)}$ $\rightarrow T(n-2) = T(n-3) + 1$
$T(n) = T(n-3) + 3$

Quindi riscrivendo l'equazione ad un generico passo k si ottiene: $T(n) = T(n-k) + k$
Il caso base equivale a T(1) = 1, allora si pone n-k= 1, da cui k= n-1, ottenendo così:
$T(n) = \mathbf{T(n-k)} + k = \mathbf{T(1)} + n - 1 = \mathbf{1} + n - 1 = n$

3.1.8 Analisi dei costi di confronto dell'InsertionSort

$$\begin{cases} C(n) = 1 + C(n-1) + n - 1 = C(n-1) + n & \text{passo ricorsivo} \\ C(1) = 1 & \text{passo base} \end{cases}$$

Il numero di confronti C(n) con un input n è dato dalla somma di 1, che equivale al costo del confronto del primo if nella funzione *ordina*, con C(n-1) che è la chiamata ricorsiva di *ordina*, ed infine n-1 sono il numero di confronti che vengono effettuati nel ciclo while di *infila* in quanto la funzione *ordina* chiama *infila* passando a parametri la collezione di dimensione n-1. Per quanto riguarda il passo base, questo è ovvio infatti se passiamo alla funzione *ordina* un input di dimensione 1, ultimo = primo, questo significa che effettua solo il confronto del primo if terminando così la funzione.

Srotolamento dell'equazione: $C(n) = C(n-1) + n$

Si sviluppa il termine $\mathbf{C(n-1)}$ $\rightarrow C(n-1) = C(n-2) + n - 1$
$C(n) = C(n-2) + (n-1) + n$

Si sviluppa il termine $\mathbf{C(n-2)}$ $\rightarrow C(n-2) = C(n-3) + n - 1$
$C(n) = C(n-3) + (n-2) + (n-1) + n$

Da cui si ottiene:

$$C(n) = C(n-k) + \sum_{i=0}^{k-1} (n-i)$$

Il caso base corrisponde a C(1) = 1, allora si pone n-k= 1 da cui si ottiene:

$$C(n) = C(1) + \sum_{i=0}^{n-2} (n-i) = 1 + \sum_{i=0}^{n-2} (n-i)$$

La sommatoria scritta in modo esplicito corrisponde a:
$n + n - 1 + n - 2 + n - 3 + n - 4 + \ldots + n - (n-3) + n - (n-2) =$
$n + n - 1 + n - 2 + n - 3 + n - 4 + \ldots + 3 + 2$
Si può osservare come equivalga alla somma dei primi n numeri a partire da 2, ovvero:

$$\sum_{h=2}^{n} h$$

Si può notare come questa sia molto simile alla *Formula di Gauss* per la somma dei primi n numeri naturali:

$$\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$$

Infatti, mettendole in relazione si ottiene che:

$$1 + \sum_{h=2}^{n} h = \sum_{k=1}^{n} k = \frac{n(n+1)}{2}$$

Riprendendo i costo dei confronti dell'InsetionSort, si ottiene che:

$$C(n) = 1 + \sum_{i=0}^{n-2}(n-i) = 1 + \sum_{h=2}^{n} h = 1 - 1 + \frac{n(n+1)}{2} = \frac{n(n+1)}{2}$$

Quindi in conclusione possiamo dire che il numeri di confronti C(n) è esattamente n^2, ovvero $\mathbf{C(n) \in \Theta(n^2)}$.

3.1.9 Analisi dei costi di assegnazione dell'InsertionSort

$$\begin{cases} A(n) = A(n-1) + n & \text{passo ricorsivo} \\ A(1) = 1 & \text{passo base} \end{cases}$$

Il numero delle assegnazioni dell'InsertionSort è dato dalla somma di A(n-1), corrispondente alla chiamata ricorsiva di *ordina* ed n, che equivale al numero di assegnazioni effettuate in *infila*. Il passo base è banale, infatti, il numero di assegnazioni che vengono effettuate su una collezione di dimensione unitaria è costante.

Srotolamento dell'equazione: $A(n) = A(n-1) + n$

Si sviluppa il termine $\mathbf{A(n-1)}$ $\rightarrow A(n-1) = A(n-2) + n - 1$
$A(n) = A(n-2) + (n-1) + n$

Si sviluppa il termine $\mathbf{A(n-2)}$ $\rightarrow A(n-2) = A(n-3) + n - 2$
$A(n) = A(n-3) + (n-2) + (n-1) + n$

Quindi come per il numero dei confronti, il numero delle assegnazioni è pari a:

$$A(n) = A(n-k) + \sum_{i=0}^{k-1}(n-i) = 1 + \sum_{i=0}^{n-2}(n-i) = 1 + \sum_{h=2}^{n} h = 1 - 1 + \frac{n(n+1)}{2} = \frac{n(n+1)}{2}$$

Quindi in conclusione possiamo dire che il numero di assegnazioni A(n) è esattamente n^2, ovvero $\mathbf{A(n) \in \Theta(n^2)}$.

3.1.10 Analisi dei costi di memoria dell'InsertionSort

Per quando concerne l'utilizzo della memoria dell'InsertionSort è possibile dire che la dimensione dell'array non varia durante l'esecuzione del programma. Quindi il costo in memoria è dato dalla dimensione dell'array n, sommato alle n variabili di supporto che vengono adoperate, quindi il numero di bit occupati è:
$n\ caselle + costante + n\ variabili$
$k_1 \cdot n + k_2 + k_3 \cdot n$
$k \cdot n + k_2 \ \ bit$
Dove $k_1,\ k_2,\ k_3$ indicano rispettivamente il numero di bit di ogni casella dell'array, delle costanti e di ogni variabile di supporto.

3.2 MergeSort

3.2.1 Metodo Divide et Impera

Molti degli algoritmi più utilizzati sono ricorsivi, ciò implica che per risolvere un determinato problema, questi chiamano sé stessi in modo ricorsivo, una o più volte. Questa tipologia di algoritmi generalmente adotta un approccio divide et impera: si suddivide il problema in sotto problemi, che sono simili all'originale, ma di dimensioni più piccole, si risolvono in modo ricorsivo e si combinano le soluzioni per costruire la soluzione del problema originale. Il paradigma prevede tre passi a ogni livello di ricorsione:

1. **Divide**. Il problema viene diviso in un certo numero di sotto problemi, che sono istanze più piccole dello stesso problema (input suddiviso in frammenti);
2. **Impera**. I sotto problemi vengono risolti in modo ricorsivo, quando essi hanno una dimensione sufficientemente piccola vengono risolti direttamente;
3. **Combina**. Le soluzioni dei sotto problemi vengono combinate per generare la soluzione del problema originale.

Il MergeSort è un algoritmo efficiente la cui complessità temporale per l'ordinamento di un intero array di dimensione n è solamente $\mathbf{n \cdot log(n)}$. L'algoritmo è conforme al paradigma Divide et Impera che viene sviluppato come segue:

1. **Divide**. Divide la sequenza degli n elementi da ordinare in due sotto-sequenze di n/2 elementi ciascuna;
2. **Impera**. Ordina le due sotto-sequenze in modo ricorsivo;
3. **Combina**. Fonde le due sotto-sequenze ordinate per generare la sequenza ordinata finale.

3.2.2 MergeSort: equazione di ricorrenza

$$\begin{cases} T(n) = 2 \cdot T(\frac{n}{2}) + O(n) & \text{passo ricorsivo} \\ T(1) = 1 & \text{passo base} \end{cases}$$

3.2.3 PseudoCodice MergeSort

```
MergeSort(A, p, r){
    if p >= r
        return;
    else{
        q = (p + r)/2
        MergeSort(A, p, q)
        MergeSort(A, q+1, r)
        Merge(A, p, q, r)
    }
}
Merge(A, p, q, r){
    nL = q − p + 1
    nR = r − q
    L[0 : nL−1]
    R[0 : nR−1]
    for i = 0 to nL−1
        L[i] = A[p+i]
    for j = 0 to nR−1
        R[j] = A[q+j+1]
    i = 0
    j = 0
    k = p
    while i < nL && j < nR
        if L[i] < R[j]
            A[k] = L[i]
            i = i + 1
        else
            A[k] = R[j]
            j = j + 1
        k = k + 1
    while i < nL
        A[k] = L[i]
        i = i + 1
        k = k + 1
    while j < nR
        A[k] = R[j]
        j = j + 1
        k = k + 1
}
```

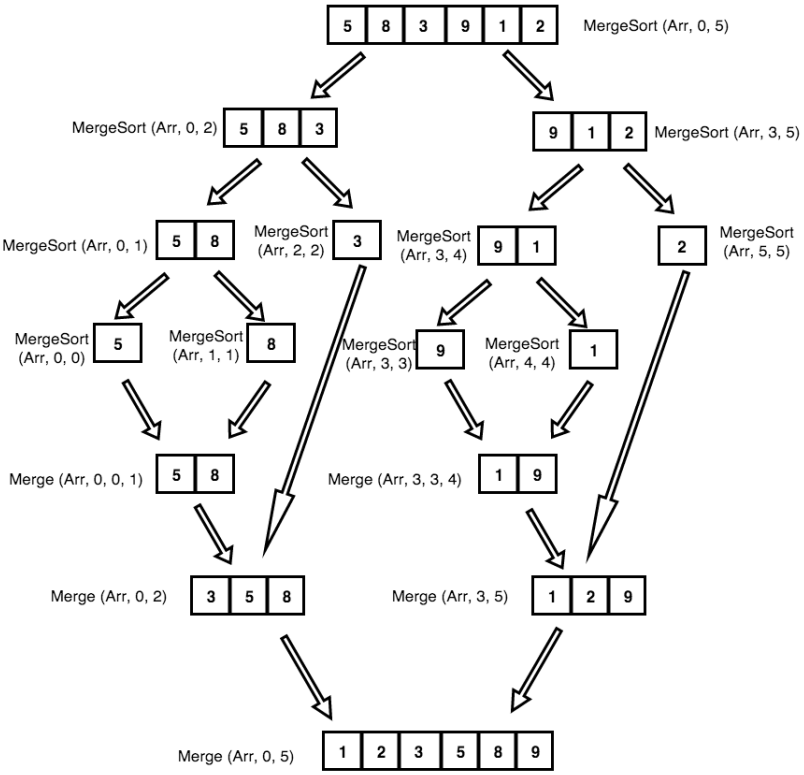
3.2.4 Codice C MergeSort

```
void Merge(array* a, int p, int q, int r){
    int nL= q-p+1; //num elementi nel sottoarray di sinistra
    int nR= r-q; //num elementi nel sottoarray di destra
    int* L= (int*)malloc(sizeof(int)*nL);
    int* R= (int*)malloc(sizeof(int)*nR);
    for(int i = 0; i < nL; i++)
        L[i] = a->arr[p + i];
    for(int j = 0; j < nR; j++)
        R[j] = a->arr[q + j + 1];
    int i=0, j=0, k=p;

    while(i<nL && j<nR){
        if(L[i] < R[j]){
            a->arr[k]= L[i];
            i++;
        }
        else{
            a->arr[k]= R[j];
            j++;
        }
        k++;
    }
    while(i<nL){
        a->arr[k]= L[i];
        i++;
        k++;
    }
    while(j < nR){
        a->arr[k]= R[j];
        j++;
        k++;
    }
}
void mergeSort_aux(array* a, int p, int r){
    if(p>=r)
        return;
    else{
        int q= (p+r)/2;
        mergeSort_aux(a, p, q);
        mergeSort_aux(a, q+1, r);
        Merge(a, p, q, r);

    }
}
void mergeSort(array *a) {
    mergeSort_aux(a, 0, a->size-1);
}
```

3.2.5 Esempio MergeSort



Immaginando di ricevere in input un array di sei interi non ordinati A= [5 8 3 9 1 2], a partire dal quale si ottengono due array A_1= [5 8 3] e A_2=[9 1 2], che andranno ulteriormente divisi a metà A_3=[5 8] e A_4=[3], A_5=[9 1] e A_6=[2], che andranno ulteriormente divisi a metà A_7=[5], A_8=[8], A_9=[9], A_10=[1] e A_6=[2]. Una volta diviso l'input in parti uguali fino a giungere a sequenze indivisibili, si confrontano gli elementi appartenenti a ciascun sotto array per poi effettuare l'operazione di Merge e quindi fondere tutti i sotto array in maniera ordinata.

3.3 QuickSort

QuickSort è un efficiente algoritmo di ordinamento divide et impera:

1. **Divide:** partiziona l'array in due sotto array.
2. **Impera:** richiama ricorsivamente QuickSort sui due sotto array

La sua complessità temporale dipende dal bilanciamento del partizionamento, nel caso sia bilanciato è $\mathbf{O(n \cdot log\ n)}$, ovvero quando la selezione del pivot divide l'array in input in due sotto array esattamente uguali o quasi, mentre quella nel caso peggiore è $\mathbf{O(n^2)}$, ovvero quando è sbilanciato quindi quando c'è una grande differenza in termini di dimensione tra i sotto array. Il QuickSort procede nella sua esecuzione per step:

1. **Scelta del pivot:** si sceglie come pivot l'ultimo elemento dell'array;
2. **Partizionamento:** si ordina l'array in modo che tutti gli elementi minori del pivot siano a sinistra e tutti quelli maggiori a destra;
3. **Ricorsione:** si richiama ricorsivamente sul sotto-array di sinistra, detto parte bassa, di indici nell'intervallo [0, q-1] e sul sotto-array di destra, detto parte alta, di indici [q+1, n-1].

Si può notare come quando l'array viene partizionato l'elemento in posizione A[q] (equivalente al pivot) si trovi nella posizione definitiva dell'array ordinato, nel sotto-array di sinistra si troveranno tutti gli elementi minori del pivot e nel sotto-array di destra tutti quelli maggiori.

3.3.1 Analisi dei costi del QuickSort

Come detto il costo del QuickSort nel caso peggiore è $\mathbf{O(n^2)}$, mentre nel caso medio e nel caso migliore è di $\mathbf{O(n \cdot log\ n)}$. Il costo in memoria dell'algoritmo invece è $\mathbf{O(n)}$ in quanto non viene allocato nuovo spazio in memoria per i sotto-array, bensì si richiama ricorsivamente sullo stesso. Questa è una differenza sostanziale che c'è con il MergeSort, infatti il suo costo in memoria dipende dal numero di frammenti che si ottengono dall'array iniziale.
Il caso peggiore del QuickSort è quando l'array da ordinare è già ordinato o è ordinato in ordine inverso, o quando tutti gli elementi sono uguali. In questi casi, il pivot scelto ad ogni passaggio divide l'array in modo molto sbilanciato: un sotto-array è vuoto (o contiene un solo elemento), mentre l'altro sotto-array contiene tutti gli altri elementi. Di conseguenza, la profondità della ricorsione è proporzionale alla lunghezza dell'array, il che porta a un tempo di esecuzione quadratico, ovvero $O(n^2)$.
Il QuickSort è di per se un algoritmo efficiente, però può essere migliorato abbassando la probabilità di incontrare dei casi sfavorevoli, quindi evitando di incontrare casi di sbilanciamento nei due sotto-array. Per fare questo si utilizza una versione randomizzata del QuickSort, in cui la scelta del pivot avviene in modo pseudo-casuale.

3.3.2 Equazione di ricorrenza QuickSort

$$\begin{cases} T(n) = O(1) & se\ n \leq 1 \ \text{ passo base} \\ T(n) = T(k) + T(n-k-1) + O(n) & se\ n > 1 \ \text{ passo ricorsivo} \end{cases}$$

Dove T(k) rappresenta il tempo di esecuzione del QuickSort per il sotto array con k elementi, T(n-k-1) rappresenta il tempo di esecuzione del QuickSort per il sotto array con n-k-1 che non includono il pivot scelto, O(n) rappresenta il tempo necessario per effettuare la partizione dell'array in base al pivot, che richiede un tempo lineare proporzionale alla dimensione dell'array. Il passo base dell'equazione di ricorrenza del QuickSort è quando l'array da ordinare contiene zero o un solo elemento, in cui il tempo di esecuzione è costante.

3.3.3 Pseudo Codice QuickSort

```
Quicksort(A, p, r){
    if p < r
        q = Partiziona(A, p, r) //restituisce l'indice q del pivot
        //eseguo la ricorsione sulla parte bassa e la parte alta
        QuickSort(A, p, q−1)
        QuickSort(A, q+1, r)
}
Partiziona(A, p, r){
    x = A[r] //scelgo come pivot l'ultimo elemento della collezione
    i = p−1
    for j = p to r−1
        if A[j] <= x
            i = i+1
            Scambia A[i] con A[j]
    Scambia A[i+1] e A[r]
    return i+1
}
```

3.3.4 Codice C QuickSort

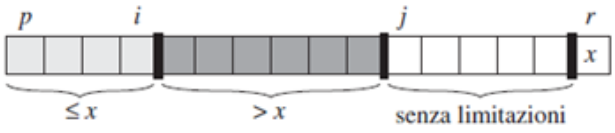
```
int partiziona(array *a, int low, int high) {
    int pivot = a->arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (a->arr[j] <= pivot) {
            i = i + 1;

            //Scambia a[i] con a[j]:
            int temp = a->arr[i];
            a->arr[i] = a->arr[j];
            a->arr[j] = temp;
        }
    }
    //Scambia a[i+1] con a[high]:
    int temp = a->arr[i+1];
    a->arr[i+1] = a->arr[high];
    a->arr[high] = temp;

    return i + 1; //Restituisce il pivot
}
void quickSort_aux(array *a, int low, int high) {
    if (low < high) {
        int pivot = partiziona(a, low, high); //Restituisce l'indice q del pivot

        //Eseguo la ricorsione sulla parte bassa e quella alta dell'array
        quickSort_aux(a, low, pivot-1);
        quickSort_aux(a, pivot+1, high);
    }
}
void quickSort(array *a) {
    quickSort_aux(a, 0, a->size-1);
}
```

3.3.5 Invariante di ciclo QuickSort



L'invariante di ciclo è una proprietà che rimane vera ad ogni iterazione di un ciclo in un algoritmo. È uno strumento utile per dimostrare la correttezza e la terminazione di un algoritmo. Per l'algoritmo QuickSort, l'invariante di ciclo può essere definita come segue: ad ogni iterazione del ciclo è possibile individuare tre sottoinsiemi di elementi dell'array tali da rispettare le seguenti condizioni.

1. Se $\mathbf{p} \leq \mathbf{k} \leq \mathbf{i}$, allora $\mathbf{A}[\mathbf{k}] \leq \mathbf{x}$
2. Se $\mathbf{i} + \mathbf{1} \leq \mathbf{k} \leq \mathbf{j} - \mathbf{1}$, allora $\mathbf{A}[\mathbf{k}] > \mathbf{x}$
3. Se $\mathbf{k} = \mathbf{r}$, allora $\mathbf{A}[\mathbf{k}] = \mathbf{x}$

L'invariante di ciclo deve valere per le tre fasi del ciclo:

Inizializzazione: All'inizio del ciclo l'indice i è impostato a p-1 e l'indice j è impostato a p. Questo significa che non ci sono elementi nei sotto vettori definiti da i e j. Quindi, l'invariante di ciclo è vera.

Mantenimento: Se l'elemento corrente A[j] è minore o uguale al pivot, incrementiamo i e scambiamo A[i] con A[j]. Questo assicura che tutti gli elementi a sinistra del pivot siano minori o uguali al pivot, e tutti gli elementi a destra del pivot siano maggiori. Così l'invariante di ciclo è mantenuta.

Terminazione: Alla fine del ciclo, scambiamo il pivot con l'elemento immediatamente a destra del sotto vettore degli elementi minori o uguali al pivot. Questo posiziona il pivot nel suo posto definitivo nell'array ordinato. A questo punto, l'invariante di ciclo garantisce che tutti gli elementi a sinistra del pivot non sono più grandi del pivot e tutti gli elementi a destra del pivot non sono più piccoli. Quindi, l'array A[p...r] è partizionato.

3.4 SelectionSort

Il SelectionSort è uno degli algoritmi di ordinamento più semplici. L'algoritmo procede per due step:

1. Trova il min dell'array e lo scambia con l'elemento in prima posizione
2. Reitera il punto 1 sull'array di dimensione n-1

Il costo di questo algoritmo è $\mathbf{\Theta(n^2)}$.

3.4.1 Codice C SelectionSort

```
int cercaMin(int primo, int ultimo){ // restituisce l'indice dell'elemento min
    int m = A[ultimo];
    int j = ultimo;
    for(int i = primo; i<ultimo; i++){
        if(A[i] < m){
        // caso peggiore: ad ogni ricorsione si entra nel ciclo e si aggiorna il minimo
            m = A[i];
            j = i;
        }
    }
    return j;
}
selection_sort (){
    for(int i = 0; i<n-1; i++){
        int temp = A[i];
        int j = cercaMin(i, n-1);
        A[i] = A[j];
        A[j] = temp;
    }
    return;
}
```

3.4.2 Analisi dei costi SelectionSort

Il numero di confronti di *cercaMin* è:

1. Alla prima iterazione, a *cercaMin* vengono passati i seguenti parametri: *primo = 0* e *ultimo = n-1*. Il numero di iterazioni che eseguirà il for equivale a calcolare *ultimo-primo*. In questo caso, quindi, vengono eseguiti n-1 confronti.

2. Alla seconda iterazione *primo = 1, ultimo = n-1*. Il numero di iterazioni *(ultimo-primo)* equivale ad n-2.
   ...
   ...

All'iterazione n-2, nella funzione *selection sort* vengono passati a *cercaMin* $i = n-2$ e $n-1$. Il numero di confronti è $(n-1)-(n-2) = n-1-n+2 = 1$ Allora è possibile dire che il numero di confronti è pari a:

$$C(n) = \sum_{i=1}^{n-1} i = \frac{(n-1)(n-1+1)}{2} = \frac{n(n-1)}{2} \in O(n^2)$$

.

3.5 CountingSort

Il counting sort è un algoritmo di ordinamento che suppone che ciascun elemento da ordinare sia un numero intero. Inizialmente si scansiona l'array per trovare il valore massimo e si alloca un array temporaneo di dimensione max+1 caselle, dove max è il valore massimo presente nell'array. Successivamente si scorre l'array originario e per ogni elemento si riempie la casella dell'array temporaneo, che ha indice corrispondente pari all'elemento, con il numero di occorrenze dell'elemento. Ad esempio, nell'array da ordinare ci sono due numeri 3, allora nell'array temporaneo, nella casella con indice 3 inserisci il numero 2. Infine si scorre l'array temporaneo e si inseriscono nell'array originario i numeri relativi agli indici delle caselle dell'array temporaneo occupate da un numero diverso da 0.

Esempio:
Ricevendo in input l'array $A =< 1, 5, 3, 9, 0 >$
A-temp:

index	0	1	2	3	4	5	6	7	8	9	10
valore	1	1	0	1	0	1	0	0	0	1	0

Il CountingSort è un algoritmo in cui non vengono effettuati confronti tra elementi dell'Array.

Ricorda che si definisce confronto tra elementi dell'array quando viene inserita, ad esempio, la condizione $if(A[i] > A[i+1])$. Invece non viene considerato un confronto la seguente condizione:

```
if(A[i] > max)
    max = A[i];
```

Infatti, nel primo caso sto confrontando direttamente due valori di indici i e i+1, invece nel secondo caso anche se max assume un valore interno all'array è come se venisse effettuato un confronto tra A[i] e una variabile. Tecnicamente il tempo impiegato dalla CPU nel secondo caso è inferiore rispetto al primo in quanto non deve ricercare ogni volta il valore dell'array in

posizione i e i+1.
Stabilire il costo del CountingSort potrebbe risultare una pratica complicata in quanto non necessariamente per $n \to \infty$ la dimensione dell'array temporaneo tende ad infinito, in quanto come abbiamo detto tale dimensione dipende dal valore del max dell'array da ordinare. In generale possiamo dire quindi che il costo di questo algoritmo è $\mathbf{\Theta(n) + \Theta(max)}$. Infatti il primo costo lineare è dato dalla scansione dell'array iniziale di dimensione n e il secondo è il costo lineare dato dalla scansione dell'array temporaneo di dimensione max+1. Nonostante ciò, è comunque possibile migliorare anche se di poco questo costo, infatti è sufficiente trovare oltre al max, anche il min dell'array così facendo è possibile risparmiare delle iterazioni generate dalla porzione dell'array temporaneo non utilizzato.

Esempio: Ricevendo in input l'array $A =< 10, 14, 11, 12, 12 >$
In questo caso max = 14 e min = 10, allora andiamo a considerare la porzione dell'array che parte dall'indice 10 e termina con indice max.
A-temp:

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
valore	0	0	0	0	0	0	0	0	0	0	1	1	2	0	1

Così facendo si andrà ad iterare non su un array di dimensione max+1, bensì su un array di dimensione $\mathbf{max - min + 1}$. Il costo allora diventa: $\mathbf{\Theta(n) + \Theta(max - min)}$

3.6 BucketSort

Il BucketSort è un algoritmo di ordinamento che suppone che ogni elemento della collezione di lunghezza n sia stato normalizzato per essere compreso nell'intervallo [0, 1]. Come dice il nome il BucketSort utilizza i bucket, ovvero dei contenitori dove si possono inserire degli elementi. In particolare, immaginiamo di dividere l'intervallo [0, 1] in n sotto intervalli tutti della stessa dimensione e poi di distribuire gli n numeri dell'input nei bucket. Quello che si otterrà non sarà una distribuzione uniforme, infatti non tutti i bucket dopo l'inserimento dei valori avranno lo stesso numero di valori. Si procede con l'ordinamento dei singoli bucket con un qualsiasi algoritmo di ordinamento e infine si ricompone l'array ordinato. Il costo del BucketSort è $\Theta(n)$. Infatti, il costo di ordinamento dei singoli bucket è costante in quanto la dimensione di essi è costante, quindi il costo di questo algoritmo si riduce a scansionare l'array da ordinare e svolgere i vari confronti per capire a quale bucket assegnare il valore i-esimo dell'array.

3.7 HeapSort

3.7.1 Definizione di Heap

Un Heap è una struttura dati composta, basata su alberi binari, ovvero alberi che possono avere al massimo due figli generalmente denominati "sinistro" e "destro". Queste strutture possono essere rappresentate mediante array i cui elementi sono i nodi dell'albero. Un array A che rappresenta un heap ha due attributi specifici: **A.length** che indica il numero di elementi dell'array e **A.heapSize** che indica il numero di elementi dell'array che appartengono all'heap. In generale si ha che $1 \leq A.heapSize \leq A.lenght$. Per ogni elemento i dell'Array, ad eccezione della radice che è l'elemento A[1], è possibile determinare il padre e i suoi due figli, in particolare:

```
Parent(i)
    return ⌊i/2⌋
Left(i)
    return 2i
```

```
Right(i)
    return 2i + 1
```

Esistono due tipi di heap binari: **max_heap** e **min_heap**. Il max_heap ha la caratteristica che la radice di ogni sottoalbero è il max dei nodi del sottoalbero, in particolare si ha che per ogni nodo i che non sia la radice: **A[Parent(i)]** > **A[i]**, ovvero il padre è maggiore del figlio. Viceversa, un min_heap ha la caratteristica che la radice di ogni sottoalbero è il min dei nodi del sottoalbero e, in particolare, si ha che per ogni nodo i che non sia la radice: **A[Parent(i)]** < **A[i]**.

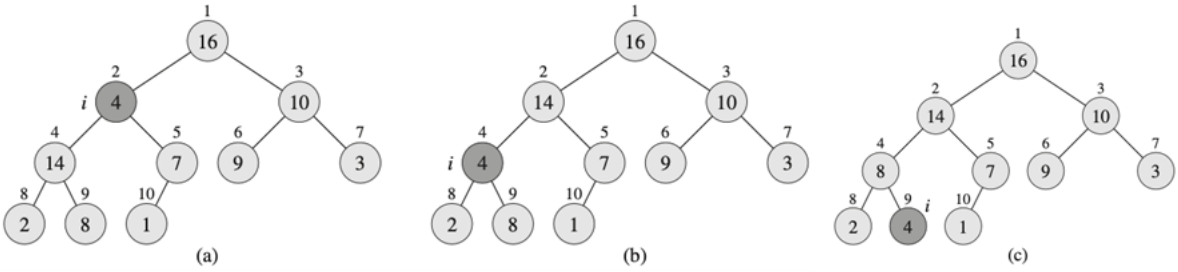
Nell'HeapSort vengono adoperate diverse funzioni applicate a dei max_heap, in particolare:

1. Max_Heapify che, eseguita in un tempo **O(log n)**, conserva le proprietà di un max_heap;

```
Max_Heapify (A, i){
    l= left(i)
    r= right(i)
    if l <= A.heapSize AND A[l] > A[i]
        largest = l
    else
        largest = i
    if r <= A.heapSize AND A[r] > A[largest]
        largest = r
    if largest != i
        Scambia A[i] e A[largest]
        Max_Heapify(A, largest)
}
```

Max_Heapify assume che venga chiamata su dei max_heap, ed ha il compito di conservare la proprietà di un max_heap, ovvero che ogni nodo è maggiore rispetto ai figli.

Funzionamento. Quando viene chiamata questa funzione, si assume che i sottoalberi con radici in **left(i)** e **right(i)** siano max_heap, mentre non è certo che il padre **i** sia effettivamente più grande dei figli. Vengono salvati gli indici del figlio sinistro e del figlio destro, dopodiché si procede a verificare innanzitutto se questi indici sono nei limiti dell'array e se viene rispettata la condizione A[i] > A[l] e A[r] (condizione di un max_heap), se così non fosse verrebbe salvato nella variabile **largest** l'indice del figlio maggiore e verrebbero scambiati A[largest] e il padre. Così facendo la condizione di un max_heap sarebbe verificata per la terna appena considerata, tuttavia potrebbe non essere verificata nel sottoalbero che ha come radice il nodo appena scambiato, allora si procede a richiamare ricorsivamente la funzione sull'indice di largest, ovvero il nodo appena scambiato. Dunque, in breve, fa scendere il valore A[i] nel max_heap in modo che il sottoalbero con radice di indice i diventi un max_heap.

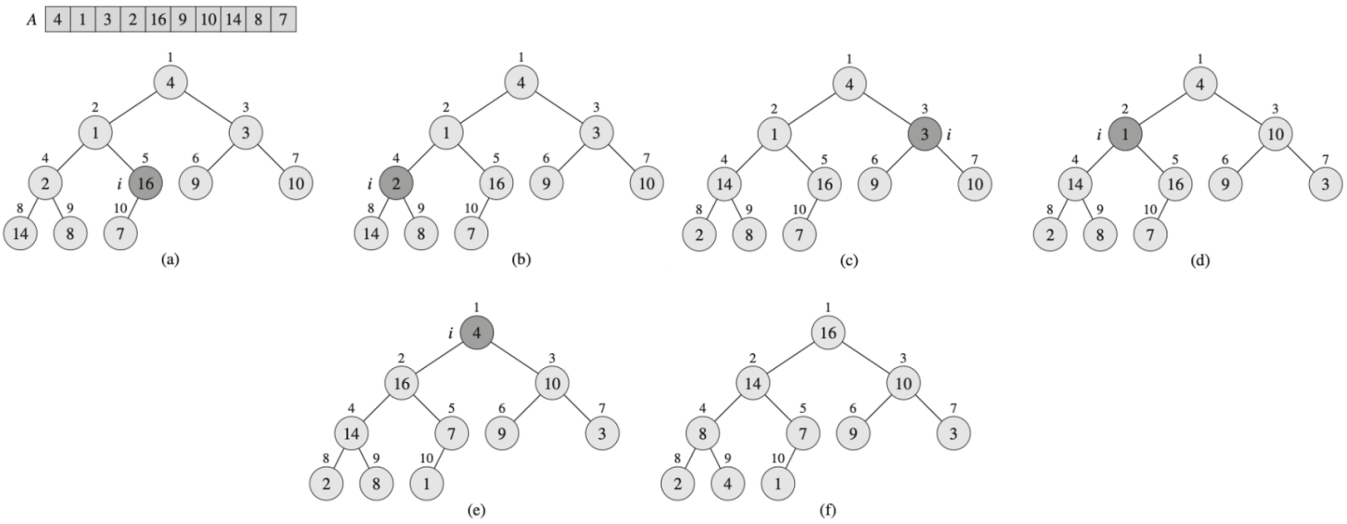


Costo. Si potrebbe indicare il suo costo con **O(h)** dove h è il tempo di esecuzione di Max_Heapify in un nodo di altezza h.

2. Build_Max_Heap che, eseguita in un tempo lineare, genera da un array di input non ordinato un max_heap;

```
Build_Max_Heap(A, n){
    A.heapSize= n
    for i= n/2 down to 1
        Max_Heapify(A, i)
}
```

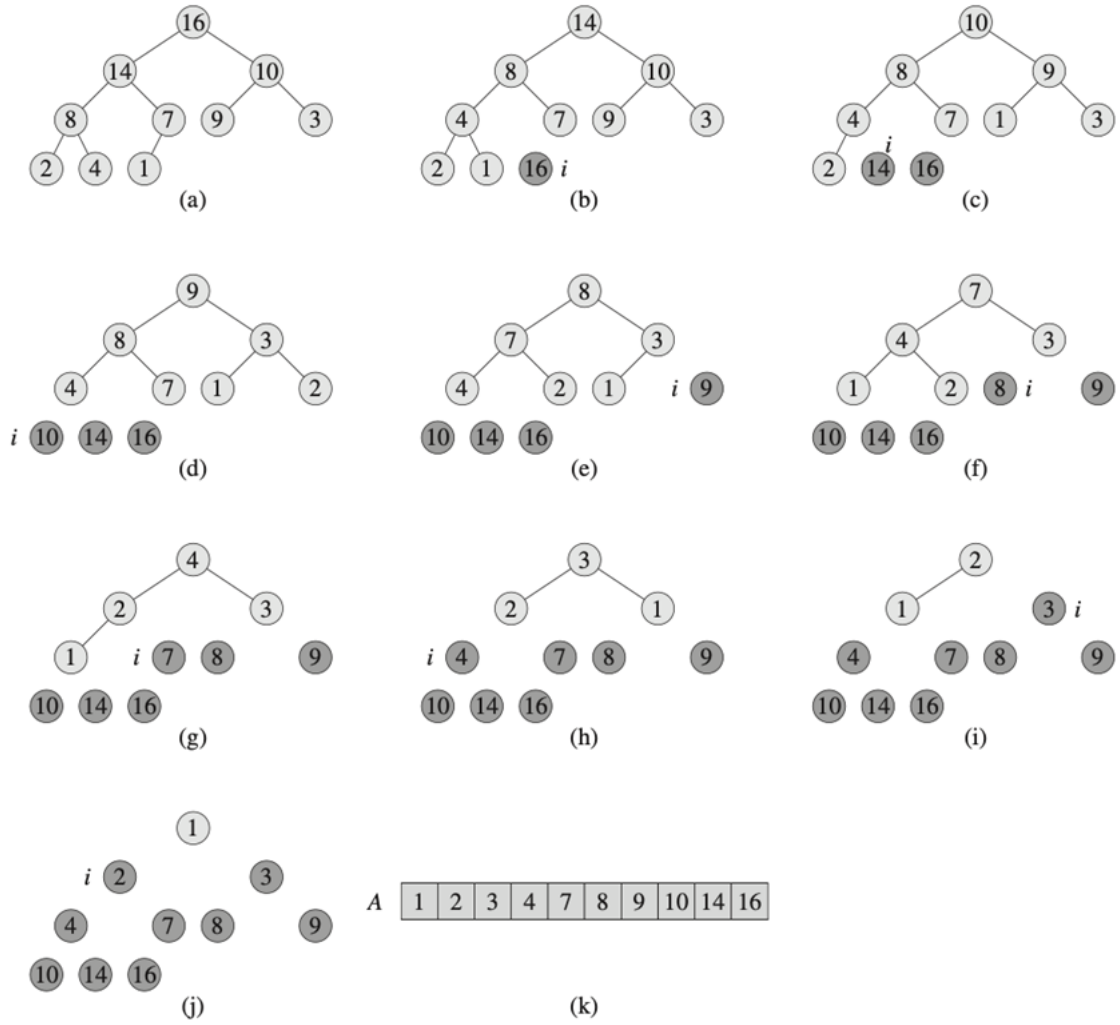
Funzionamento. La funzione Max_Heapify può essere utilizzata in modo invertito, ovvero dal basso verso l'alto (bottom-up), per convertire un array di input con n = A.length in un max_heap. È possibile utilizzare la funzione Max_Heapify poiché gli ultimi elementi di ciascun sottoalbero possono essere considerati foglie dell'albero, dunque si parte da esse per salire verso la radice. Build_Max_Heap si limita a richiamare iterativamente Max_Heapify su A passando l'indice i che varia dalla metà dell'array decrementato fino ad 1, così facendo fa salire nell'albero i valori più grandi e fa scendere quelli più bassi.



2. Heap_Sort che, eseguita in un tempo O(n log n), ordina un array.

```
HeapSort(A, n){
    Build_Max_Heap(A, n)
    for i=n down to 2
        Scambia A[1] e A[i]
        A.heapSize= A.heapSize −1
        Max_Heapify(A, 1)
}
```

Funzionamento. La funzione HeapSort chiama inizialmente Build_Max_Heap(A, n) per trasformare A in un max_heap. Dunque, se per definizione l'elemento più grande dell'array è memorizzato nella radice A[1], per ordinare in modo crescente l'albero, basta scambiare la radice con l'elemento in posizione finale, ovvero A[n]. Dopodiché si decrementa A.heapSize, in modo tale da non considerare più l'elemento in ultima posizione, in quanto si trova nella sua posizione definitiva. Per ottenere un max_heap basta chiamare Max_Heapify(A, 1) per far tornare il max alla radice. Questo ciclo viene ripetuto n-1 volte. Build_Max_Heap impiega **O(n)**, mentre Max_Heapify **O(log n)**, in totale il costo ammonta a **O(n log n)**.



3.7.2 Code di max-priorità

Una delle applicazioni più efficienti e diffuse dell'heap è la coda di priorità. Una coda di priorità è una struttura dati che serve a mantenere in ordine un insieme S di elementi, rispetto ad un valore associato a ciascuno di essi, detto chiave. Una coda di max-priorità supporta le seguenti operazioni:

1. **Insert(S, x)** inserisce l'elemento x nell'insieme S;
2. **Maximum(S)** restituisce l'elemento di S con la chiave più grande;
3. **Extract_Max(S)** rimuove e restituisce l'elemento di S con la chiave più grande;
4. **Increase_Key(S, i, k)** cerca l'elemento x dell'insieme S che si trova in posizione i per aggiornarne il valore della chiave, dopodiché controlla che l'elemento sia nella corretta posizione nell'array, in base alla chiave k.

Per rappresentare una coda di priorità si utilizza un heap, in questo caso si ha bisogno di due array: un array di chiavi che definisce le priorità e un array di oggetti che sono gli elementi di S. In particolare, ad ogni elemento in posizione i dell'array di oggetti è associata la rispettiva chiave in posizione i nell'array delle chiavi. Bisogna notare che la procedura Max_Heap_Maximum implementa l'operazione Maximum, mentre Max_Heap_Extract_Max implementa l'operazione Extract_Max, infine Max_Heap_Increase_Key implementa Increase_Key. Nota come in quest'ultimo caso l'elemento non venga tolto realmente dall'array, ma solo "ignorato" decrementando la dimensione dell'heap. Quando modifico la chiave di x, è necessario

riordinare A come un max_heap, quindi, deve essere rispettata la condizione **A[Parent(i)]** > **A[i]**, appena la condizione del while non è verificata significa che è già un max_heap. In caso contrario scambia il padre con A[i] e di conseguenza scambia anche l'indice. Nota bene come viene controllato solo il valore del padre e non dei figli, essendo infatti un max_heap se inserisco al posto di un nodo i un valore più grande la condizione del max_heap è verificata sicuramente verso i figli del nodo i ma non è certo che è rispettata con il padre.

```
Max_Heap_Maximum(A){
    if A.heapSize < 1
        Error "Underflow heap"
    return A[1]
}
Max_Heap_Extract_Max(A){ //restituisce il max delle chiavi e lo toglie dall'albero
    max= Max_Heap_Maximum(A)
    A[1]= A[A.heapSize] // scambia la radice (max dell'albero) con l'ultima foglia
    A.heapSize= A.heapSize −1
    Max_Heapify(A, 1) // riordino l'albero per farlo tornare un max_heap
    return max
}
Max_Heap_Increase_Key(A, i, k){
// modifica la chiave di x in posizione i e posiziona k nella
// giusta locazione dell'heap
    if k < A[i]
        Error
    A[i]= k
    while i > 1 AND A[Parent(i)] < A[i]
        Scambio A[i] e A[Parent(i)]
        i= Parent(i)
}
```

4 I Grafi

Un grafo è una generalizzazione del concetto di albero, infatti questo è un tipo speciale di grafo. In particolare, un albero è un grafo connesso e aciclico. Questo significa che in un albero non ci sono cicli (percorsi che iniziano e finiscono nello stesso nodo) e che tutti i nodi sono collegati da esattamente un percorso. Se nei grafi da ogni vertice possono partire e giungere più archi, negli alberi ogni nodo ha al massimo un padre. Un grafo viene definito come una struttura matematica in grado di rappresentare relazioni tra oggetti. È composto da un insieme di nodi (o vertici) e un insieme di archi che collegano coppie di nodi. I nodi rappresentano entità, mentre gli archi rappresentano le relazioni tra di esse.

4.1 Definizioni dei Grafi

Def. 1. Un grafo G= (V, E) è costituito da un insieme di vertici **V** ed un insieme di archi **E** ciascuno dei quali connette due vertici detti estremi dell'arco.

Def. 2. Un grafo è **orientato** quando esiste una **relazione d'ordine** tra gli estremi degli archi, ovvero gli archi hanno un verso e una direzione definita, espressi tramite una freccia. Immaginiamo un arco come una freccia che va dalla coda alla testa, infatti il primo estremo si dice coda ed il secondo testa.

Def. 3. Un grafo si dice **denso** se il numero di archi presenti è molto vicino al numero totale di possibili archi.

Def. 4. Un **cappio** è un arco i cui estremi coincidono.

Def. 5. Un grafo orientato è **semplice** se non ci sono cappi e non ci sono due archi con gli stessi estremi. Se esistessero due archi con gli stessi estremi si definirebbe **multigrafo**.

Def. 6. Se un grafo è semplice possiamo identificare un arco con la coppia dei suoi estremi: $e = uv \in E$, in questo caso diciamo che l'arco e è incidente in u e in v.

Se il grafo è orientato, la coppia uv è ordinata. In questo caso diciamo che l'arco e esce da u ed entra in v.

Se $e = uv \in E$ diciamo che il vertice v è **adiacente** al vertice u. Se il grafo non è orientato la relazione di adiacenza è simmetrica.

Def. 7. Il **grado** $\delta(\mathbf{v})$ del vertice v è il numero di archi incidenti in v. Se il grafo è orientato abbiamo un grado entrante $\delta^-(v)$, che è il numero di archi entranti in v ed un grado uscente $\delta^+(v)$, che è il numero di archi uscenti da v.

Un **nodo pozzo** è un nodo che ha grado uscente pari a zero, $\delta^+(v) = 0$.

Def. 8. Un **percorso** π di lunghezza k dal vertice u al vertice v in un grafo G = (V, E) è una sequenza di k+1 vertici $\pi = x_0,\ x_1,\ x_2,\ \cdots,\ x_k$ tali che $x_0 = u$, $x_k = v$ e $x_{i-1}x_i \in E$ per i = 1, . . . , k.

Il percorso $\pi = x_0$ ha lunghezza k = 0.

Se $k > 0$ e $x_0 = x_k$ diciamo che il percorso è **chiuso**.

Def. 9. Un **cammino** è un percorso i cui vertici $x_0,\ x_1,\ \cdots,\ x_k$ sono tutti diversi con la possibile eccezione di $x_0 = x_k$, nel qual caso esso è un **ciclo**.

Quando esiste almeno un cammino dal vertice u al vertice v diciamo che il vertice v è **accessibile** o **raggiungibile** da u.

Def. 10. Un grafo non orientato si dice **connesso** se esiste almeno un cammino tra ogni coppia di vertici.

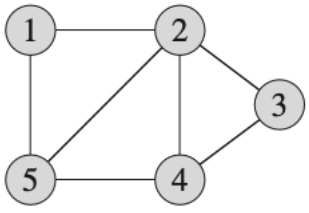
Le componenti connesse di un grafo sono le classi di equivalenza dei suoi vertici rispetto alla relazione di accessibilità. Quando si parla di componente connessa di un grafo, si fa riferimento ad un gruppo di nodi del grafo in cui esiste un percorso da qualsiasi nodo del gruppo verso qualsiasi altro nodo del gruppo. La "relazione di accessibilità" a cui si fa riferimento è semplicemente la possibilità di raggiungere un nodo da un altro attraverso uno o più archi nel grafo. Le "classi di equivalenza" a cui si fa riferimento sono insiemi di nodi che sono considerati equivalenti in termini di accessibilità. Dunque, due nodi appartengono alla stessa classe di equivalenza se e solo se sono raggiungibili tra loro attraverso archi nel grafo. Nei grafi orientati la proprietà di raggiungibilità non è simmetrica, quindi va verificata in entrambi gli estremi del cammino.

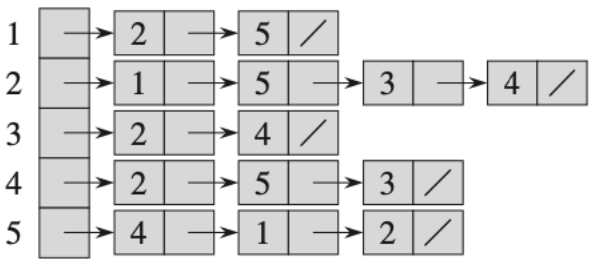
Def. 11. Un grafo orientato si dice **fortemente connesso** se esiste almeno un cammino da ogni vertice u ad ogni altro vertice v.

Def. 12. Un sottografo del grafo G = (V, E) è un grafo G' = (V', E') tale che $V' \subseteq V$ e $E' \subseteq E$.
Il sottografo di G = (V, E) indotto da $V' \subseteq V$ è il grafo G' = (V', E') tale che $E' = \{uv : uv \in E \text{ e } u, v \in V'\}$.

4.2 Rappresentazione di un grafo

Ci sono due modi per rappresentare un grafo G= (V, E): con le liste di adiacenza oppure con la matrice di adiacenza. La rappresentazione di tale grafo mediante le liste delle adiacenza utilizza liste Adj[u] con $u \in V$, in cui vengono memorizzati i vertici adiacenti al vertice u.





Nella rappresentazione di G= (V, E) mediante la matrice di adiacenza assumiamo che i vertici siano numerati da 1 fino a $|V|$, la rappresentazione è costituita da una matrice booleana A tale che:

$$\begin{cases} 1 \ se \ ij \in E \\ 0 \ se \ ij \notin E \end{cases}$$

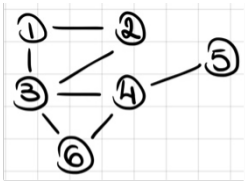
Esempio con il grafo precedente:

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

Una matrice di adiacenza di un grafo non orientato è sempre simmetrica. Per quanto concerne l'efficienza di queste due rappresentazioni, bisogna considerare che la lista di adiacenza richiede spazio in memoria pari a $|V| + |E|$, dove $|V|$ indica la cardinalità di V (il numero di vertici = il numero di puntatori alle liste collegate), mentre $|E|$ indica il numero degli elementi delle liste. La rappresentazione mediante matrice di adiacenza invece richiede spazio per una matrice di dimensione $|V|^2$ questo significa che se un grafo è denso converrà utilizzare la matrice delle adiacenze.

4.3 Visita in Ampiezza di un grafo BFS

Dato un grafo G = (V, E) e un vertice distinto **s**, detto **sorgente** la visita in ampiezza BFS (breadth-first search) ispeziona gli archi di G per visitare tutti i vertici che sono raggiungibili da s.



Si parte dal nodo sorgente, la cui scelta è arbitraria, per poi propagarsi in ampiezza; dunque, preso 1 gli attribuisco la distanza da sé stesso, ovvero zero. Al secondo passo allargo l'area fino ad includere i nodi 2 e 3, che inserisco in una coda ed incremento di uno la distanza dalla sorgente. Dopodiché espando l'area ai vicini di questi ultimi nodi visitati, così via fino a visitare tutto il grafo.

4.3.1 PseudoCodice BFS

```
BFS (G, S) {        // S = sorgente, G = grafo
    foreach u in G.V − {S} // per ogni nodo appartenente al grafo, escludendo la sorgente
        u.color = "BIANCO"
        u.d = infinity
        u.p = nil
    s.color = "GRIGIO"
    s.d = 0
    s.p = nil  // il padre della sorgente non e' nessuno
    Q = 0
    // creo la struttura coda che funziona con la regola FIFO,
    // che rappresenta la frontiera che sto visitando
    ENQUEUE (Q, S)  // inserisco nella coda la sorgente
    while Q != 0
        u = DEQUEUE (Q) // visito il nodo u e lo tolgo dalla frontiera
        foreach v in G.adj[u]   // scansiono la lista delle adiacenze del vertice u
```

```
            if v.color = "BIANCO"
                v.color = "GRIGIO"  // aggiungo alla frontiera ogni vertice v della lista di u
                v.d = u.d +1
                // la distanza di v dalla sorgente equivale alla distanza del
                // suo predecessore u incrementato di 1
                v.p = u
                ENQUEUE (Q, v)  // metto v in coda
        u.color = "NERO"
}
```

4.3.2 Codice C BFS

```
void BFS(graph* g) {
    linked_list_node * source= (linked_list_node *)g->nodes->head;
    graph_node* s= (graph_node*)source->value;
    s->status= EXPLORING;
    s->dist= 0;

    linked_list * Q= linked_list_new();
    linked_list_enqueue (Q, s);

    while( linked_list_size (Q)!=0){
        linked_list_node * node= (linked_list_node*) linked_list_dequeue (Q);
        graph_node* u= ( graph_node*)node->value;

        if( linked_list_size (u->out_edges)!=0){  // verifico che u abbia dei discendenti
            linked_list_node * current= (linked_list_node *)u->out_edges->head;
            while(current!=NULL){
                graph_edge* edge= (graph_edge*)current->value;
                graph_node* v= (graph_node*)edge->target;

                if(v->status==UNEXPLORED){
                    v->status= EXPLORING;
                    v->dist= u->dist + 1;
                    linked_list_enqueue (Q, v);
                }
                current= current->next;
            }
        }
        u->status= EXPLORED;
    }
}
```

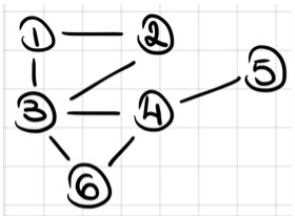
Analisi

La visita viene detta in ampiezza perché l'algoritmo espande uniformemente la frontiera; infatti, scopre tutti i vertici a distanza k dalla sorgente s prima di scoprire quelli a distanza k+1. Per mantenere traccia dell'esecuzione dell'algoritmo i vertici vengono colorati di **bianco** (vertici non raggiunti), **grigio** (vertici raggiunti nella frontiera) e **nero** (vertici raggiunti che non sono

nella frontiera). Inizialmente l'algoritmo effettua un ciclo preliminare su ogni vertice esclusa la sorgente del grafo, modificandone il colore in "BIANCO", ponendo la distanza di ogni vertice dalla sorgente ad infinity e assegnando come predecessore null. Dopodiché vengono modificati i campi della sorgente, ponendo il colore a "GRIGIO" in quanto dobbiamo visitarlo e quindi inserendolo nella frontiera e ponendo la sua distanza dalla sorgente pari a 0 ed infine aggiungo il vertice sorgente alla coda, perché, come detto, è stato aggiunto alla frontiera. In seguito, viene svolto un ciclo while sulla coda Q dove si itera su ogni nodo. Inizialmente viene richiamata la funzione **DEQUEUE** che restituisce il nodo appena rimosso dalla coda, questo sta a significare che sto visitando il nodo appena rimosso. Dopodiché effettuo un ciclo su tutti i nodi della lista delle adiacenze del nodo u (ovvero tutti i nodi che sono collegati al nodo u) aggiungendoli in lista solo se essi sono colorati di "BIANCO" ovvero che non sono presenti in coda e quindi nella frontiera, incrementando la loro distanza dalla sorgente e ponendo il loro colore a "GRIGIO" in quanto li inserisco nella frontiera. Procedendo con questo ragionamento si vanno a visitare per ampiezza tutti i vertici del grafo. La complessità dell'algoritmo BFS è $\mathbf{O}(|\mathbf{V}| + |\mathbf{E}|)$, dove $|\mathbf{V}|$ è il numero di vertici del grafo e $|\mathbf{E}|$ è il numero di archi del grafo.

Esempio

Consideriamo il seguente grafo con sorgente il nodo 1:



Per prima cosa l'algoritmo BFS inizializza tutti i nodi del grafo esclusa la sorgente. In particolare assegna ad essi il colore "BIANCO", per indicare che ancora non sono stati visitati, pone la distanza dalla sorgente pari ad infinito e inizializza a nil i loro predecessori. Dopodiché inizializza la sorgente ponendo il colore a "GRIGIO" per indicare che l'algoritmo sta visitando il nodo e pone la distanza dalla sorgente a zero. In seguito viene creata una coda inserendoci la sorgente. Quindi attualmente la coda contiene solo il vertice 1, Q = $< 1 >$. All'interno del ciclo while, che itera sulla coda, viene rimosso il primo nodo della coda (assegnato ad u), e viene scansionata la sua lista delle adiacenze, quindi nel caso del nodo 1, la lista di adiacenza sarà Adj[1] = $< 2, 3 >$. Per ogni nodo della lista di adiacenza si verifica che non sia già stato visitato (v.color = "BIANCO"), in questo caso né il nodo 2 né il nodo 3 sono stati visitati, allora per ciascuno di essi viene modificato il colore in "GRIGIO", modificata la distanza dalla sorgente che in questo caso sarà per entrambi v.d = 1 e aggiunto il coda, Q = $< 2, 3 >$, infine viene aggiornato il colore del nodo 1 a "NERO" per indicarne il termine della visita. Si passa al nodo 2, questo viene rimosso dalla coda e ne si valuta la lista di adiacenza, in questo caso Adj[2] = $< 1, 3 >$, entrambi i nodi o già sono stati visitati oppure si trovano già nella frontiera e quindi in coda, quindi questa iterazione termina ponendo il colore del nodo 2 a "NERO". All'iterazione successiva viene rimosso il nodo 3 e si valuta la sua lista di adiacenza, Adj[3] = $< 4, 6 >$, entrambi i nodi non sono stati nè visitati né si trovano nella frontiera, allora per questi ne si modifica il colore, viene posta la loro distanza dalla sorgente a v.d = 2 e vengono aggiunti in coda, Q = $< 4, 6 >$. All'iterazione successiva viene rimosso il nodo 4, la sua lista delle adiacenze è Adj[4] = $< 5, 6 >$, viene inserito solo il nodo 5, modificato il colore in "GRIGIO" e gli viene assegnata la distanza dalla sorgente pari a v.d = 3, infine viene aggiunto in coda, Q = $< 6, 5 >$. All'iterazione successiva viene valutato il 6, la cui lista delle adiacenze è Adj[6] = $< 3, 4 >$, entrambi i nodi in lista già sono stati visitati quindi l'esecuzione passa direttamente al nodo 5. La sua lista delle adiacenze è Adj[5] = $< 4 >$, ma il nodo 4 è già stato visitato, allora BFS termina così l'esecuzione.

4.3.3 Proprietà delle distanze

Sia G = (V, E), un grafo e $s \in V$ un vertice arbitrario. Allora per ogni arco $(u, v) \in E$ si ha che:

$$\mathbf{d(s, v) \leq d(s, u) + 1}$$

Dimostrazione:
Per dimostrare questa proprietà dobbiamo considerare due casi:

1. u non è raggiungibile da s
2. u è raggiungibile da s

Caso 1.
E' noto come nell'algoritmo BFS vengono inizializzate ad ∞ le distanze di tutti i nodi dalla sorgente. Se u non è raggiungibile da s, significa che non verrà mai aggiunto alla coda e la sua distanza **u.d** da s rimarrà ∞. Se u non è raggiungibile da s, non lo è neanche v allora v.d = ∞, che verifica la diseguaglianza.

Caso 2.
Se u è raggiungibile da s, significa che **u.d** viene aggiornato con la distanza da u ad s che chiameremo **d(s, u)**, se questo cammino passa già per v, allora banalmente è verificata $\mathbf{d(s, v) < d(s, u)}$, se invece il cammino non passa per v, allora aggiungendo l'arco (u, v) si ha che $\mathbf{d(s, v) = d(s, u) + 1}$, da cui segue la diseguaglianza. □

4.3.4 Proprietà del limite superiore

Sia G = (V, E) un grafo e supponiamo che BFS venga eseguita da un nodo $s \in V$ arbitrario (sorgente). Allora per ogni vertice $v \in V$, la distanza v.d calcolata da BFS è sempre tale che:

$$\mathbf{v.d \geq d(s, v)}$$

Dimostrazione:
Per dimostrare questa proprietà dobbiamo farlo per induzione.

Passo Base:
Il passo base è rappresentato dall'inizializzazione della BFS nella quale tutte le distanze dei vertici del grafo dalla sorgente sono poste ad ∞, quindi si ha banalmente che per ogni nodo u diverso da s:

$$u.d = \infty \geq d(s, u)$$

Per quanto riguarda la sorgente viene posta s.d= 0 che banalmente è uguale a d(s, s). Quindi possiamo concludere dicendo che il passo base verifica la proprietà del limite superiore.

Passo induttivo:
Supponiamo che la diseguaglianza $u.d \geq d(s, u)$ sia vera fino a prima della riga

$$\mathbf{v.d = u.d + 1}$$

dopo essere stata eseguita per ipotesi induttiva sappiamo che $\mathbf{u.d + 1 \geq d(s, u) + 1}$, dalla proprietà delle distanze abbiamo che $\mathbf{d(s, u) + 1 \geq d(s, v)}$, allora mettendo insieme tutte le disuguaglianze si ha: $\mathbf{v.d = u.d + 1 \geq d(s, u) + 1 \geq d(s, v)}$, ottenendo così $\mathbf{v.d \geq d(s, v)}$. □

4.3.5 Proprietà della coda

Si considera l'esecuzione di BFS su un grafo G = (V, E) e supponiamo che la coda Q contenga i vertici $< v_1, v_2, \ldots, v_r >$ (con v_1 inizio e v_r fine della coda), allora si ha che:

1. $v_r.d \leq v_1.d + 1$
2. $v_i.d \leq v_{i+1}.d$ per ogni i in $1, 2, \ldots, r-1$

Dimostrazione:
Per dimostrare questa proprietà dobbiamo farlo per induzione.

Passo Base:
Dopo l'inizializzazione la coda contiene solo s, allora la proprietà è verificata.

Passo induttivo:
1. Assumiamo che sia vera la seguente disuguaglianza:

$$v_r.d \leq v_1.d + 1 \leq v_2.d + 1$$

Quando v_1, viene estratto dalla coda, v_2 diventa la nuova testa e si ha banalmente

$$v_r.d \leq v_2.d + 1$$

2. Sia v_r l'ultimo elemento in coda, prima di essere stato inserito in coda è stato posto $\mathbf{v.d = u.d + 1}$, con u il primo elemento della coda ovvero v_1, quindi $\mathbf{v_r.d = v_1.d + 1}$. Ipotizziamo venga aggiunto un altro vertice tale per cui l'ultimo elemento è v_{r+1}. Si ha che $\mathbf{v_{r+1}.d = v_1.d + 1}$. Per ipotesi induttiva, inoltre, si ha che $\mathbf{v_r.d \leq v_1.d + 1}$ ma allora:

$$v_r.d \leq v_1.d + 1 = v_{r+1}.d$$

da cui:

$$\mathbf{v_r.d \leq v_{r+1}.d}$$

□

4.3.6 Correttezza della BFS

L'algoritmo BFS visita tutti i vertici raggiungibili da **s** e quando termina **v.d = d(s, v)** per ogni vertice v del grafo. Inoltre per ogni vertice $v \neq s$ raggiungibile da s:

- u = v.p $\neq$ nil
- (u, v) $\in$ E
- d(s, v) = d(s, u) + 1

Dimostrazione:
Se v non è raggiungibile da s allora dalla proprietà del limite superiore si ha che

$$v.d = d(s, v) = \infty$$

. Questo valore rimarrà invariato per tutta l'esecuzione dell'algoritmo, questo significa che il vertice v non verrà mai aggiunto in coda e quindi i vertici non raggiungibili non vengono visitati da BFS.
Se v è raggiungibile da s, per ogni vertice $v \in V$ raggiungibile da s i seguenti punti vengono effettuati una ed una sola volta:

1. a v.color viene assegnato il colore "GRIGIO"
2. a v.d viene assegnato il valore d(s, v)
3. se v è diverso da s, a v.p viene assegnato un vertice u tale che $\mathbf{d(s, v) = d(s, u) + 1}$, se $v = s$ a s.p è assegnato nil
4. il vertice v viene aggiunto alla coda Q

Se considerassimo la sorgente banalmente l'inizializzazione avviene una sola volta quindi:

- s.color = "GRIGIO"
- s.d = 0
- s.p = nil
- ENQUEUE(Q, S)

Si considera poi un insieme di vertici

$$V_k = \{v \in V \ : \ d(s, v) = k\}$$

ovvero l'insieme di vertici posti a distanza k da s. Sia $v \in V_k$, durante l'esecuzione del ciclo while la coda Q non è mai vuota e non appena il vertice v viene messo in coda i valori v.d e v.p non possono più cambiare in quanto a v.color viene assegnato il colore "GRIGIO" e per entrare nell'if è richiesto che v.color sia "BIANCO", di conseguenza se l'ordine di inserimento in coda è

$$\{s = v_0, \ v_1, ..., \ v_t\}$$

banalmente si ha che

$$v_i.d \leq v_{i+1}.d$$

che corrisponde alla proprietà della coda. Il vertice v a distanza k non può essere scoperto prima, in quanto non è adiacente a nessun vertice a distanza minore k-1, infatti a tale distanza è presente u distante da s k-1, a cui s è adiacente. □

4.4 Visita in profondità DFS

L'algoritmo DFS (deep-first search) visita in profondità un grafo, ispezionando gli archi uscenti a partire dall'ultimo vertice v scoperto. Quando tutti gli archi di v sono stati ispezionati "torna indietro" per ispezionare i restanti archi del vertice da cui v era stato scoperto.

4.4.1 Pseudo Codice DFS

```
DFS(G, s){
    foreach u in G.V
        u.color= "BIANCO"
        u.p= nil
        time= 0
    foreach u in G.V
        if u.color = "BIANCO"
            DFS_VISIT(G, u)
```

```
}
DFS_VISIT(G, u){
    time= time + 1;
    u.d= time // istante di scoperta
    u.color= "GRIGIO"

    foreach v in G.adj[u]
        if v.color = "BIANCO"
            v.p= u
            DFS_VISIT(G, v) // chiamo ricorsivamente sul primo nodo non ancora scoperto

    time= time + 1
    u.f= time // istante di completamento
    u.color= "NERO"
}
```

4.4.2 Codice C DFS

```
void DFS_visit(graph* g, graph_node* u, int* time, char* format_string){
    *time= *time + 1;
    u->status= EXPLORING;

    linked_list_node * current= g->nodes->head; // lista dei vertici del grafo
    while(current!=NULL){
        graph_node* v= (graph_node*)current->value;
        if(v->status==UNEXPLORED)
            DFS_visit(g, v, time, format_string);

        current= current->next;
    }
    *time= *time + 1;
    u->status= EXPLORED;
    u->timestamp= *time;  // timestamp di fine visita del nodo
}
void DFS(graph* g, char* format_string){
    int *time= (int*)malloc(sizeof(int));
    *time= 0;

    linked_list_node * current= g->nodes->head; // lista dei vertici del grafo
    while(current!=NULL){
        graph_node* u= (graph_node*)current->value;
        if(u->status==UNEXPLORED)
            DFS_visit(g, u, time, format_string);

        current= current->next;
    }
}
```

Analisi:

DFS pone due marcatempi su ogni vertice u, in particolare il primo identifica il tempo di scoperta (u.d) e viene assegnato quando u diventa **grigio**, mentre il secondo marcatempo identifica il tempo di completamento (u.f) ossia quando il vertice è completato e colorato di **nero**. DFS inizializza il colore di tutti i vertici a **bianco** e inizializza una variabile globale che identifica il tempo. Successivamente chiama **DFS_VISIT** sui vertici u che non sono stati ancora visitati. All'interno di DFS_VISIT viene incrementata la variabile tempo e viene assegnato al primo marcatore di u, indicandone l'istante in cui viene colorato di "GRIGIO". Dopodiché si prende la lista di adiacenza del nodo u e si scorre richiamando **ricorsivamente** DFS_VISIT sul nodo v facente parte della lista di u. Infine viene incrementata nuovamente la variabile tempo e viene assegnata al marcatore di completamento di u. La complessità di DFS è $\mathbf{O}(|\mathbf{V}| + |\mathbf{E}|)$, dove $|\mathbf{V}|$ è il numero di vertici del grafo e $|\mathbf{E}|$ è il numero di archi del grafo.

4.4.3 Teorema delle parentesi

In una qualsiasi DFS di un grafo G = (V, E) (orientato o non orientato), per ogni coppia di nodi u, v è soddisfatta una e una sola delle seguenti tre condizioni:

1. Gli intervalli [u.d, u.f] e [v.d, v.f] sono completamente disgiunti; u e v non sono discendenti uno dell'altro nella foresta della visita in profondità.
2. L'intervallo [u.d, u.f] è interamente contenuto nell'intervallo [v.d, v.f] e u è un discendente di v in un albero di visita in profondità.
3. L'intervallo [v.d, v.f] è interamente contenuto nell'intervallo [u.d, u.f] e v è un discendente di u in un albero di visita in profondità.

Dimostrazione:
Caso u.f < v.d
Sapendo che u.d < u.f e v.d < v.f, ne consegue che:

$$u.d < u.f < v.d < v.f$$

e si deduce che i due intervalli non hanno intersezione e quindi sono disgiunti. Bisogna notare che essendo i due intervalli disgiunti si ha che il vertice v viene scoperto dopo che il nodo u è stato visitato, segue che non esiste un cammino che congiunge u e v.

Caso u.f > v.d
In questo caso si ha che il vertice v viene scoperto quando u è grigio (in esplorazione), quindi la lista delle adiacenze di v viene completamente esplorata prima di riprendere l'esplorazione di u e pertanto v viene completato prima di u. □

4.4.4 Teorema del cammino bianco

In una foresta di visita in profondità di un grafo G = (V, E) (orientato o non orientato), il vertice v è discendente del vertice u se e solo se, al tempo u.d quando u viene scoperto il vertice v può essere raggiunto da u lungo un cammino che è formato esclusivamente da vertici bianchi.

Dimostrazione:
⇒) Sia v un discendente di u, sia $\mathbf{P} = \mathbf{x_0}, \mathbf{x_1}, \ldots, \mathbf{x_k}$ (con $x_0 = u$ e $x_k = v$) la sequenza dei vertici da u a v. Sapendo che x_{i+1} viene scoperto dalla lista delle adiacenze di x_i esiste l'arco

(x_i, x_{i+1}) e banalmente x_i viene visitato prima di x_{i+1}. Quindi nel cammino P quando $x_0 = u$ viene scoperto, tutti i nodi successivi ancora non sono stati scoperti e quindi sono bianchi.

⇐) Supponiamo che quando u viene scoperto esista un cammino bianco da u a v, allora essendo che v viene scoperto dopo u, dalla proprietà delle parentesi si ha che

$$(_u \ldots u)_u \ldots (_v \ldots v)_v$$

quindi sono disgiunti, oppure

$$(_u \ldots (_v \ldots v)_v \ldots u)_u$$

quindi sono discendenti.
Supponiamo per assurdo che

$$(_u \ldots u)_u \ldots (_v \ldots v)_v$$

e che quindi v non sia discendete di u. Si può assumere che v sia il primo vertice del cammino bianco che non è discendete di u. Sia $w = x_{k-1}$, il vertice che precede v nel cammino bianco. Siccome w è discendente di u per la proprietà delle parentesi si ha che

$$(_u \ldots (_w \ldots w)_w \ldots u)_u \ldots (_v \ldots v)_v$$

ma questo è assurdo in quanto v è nella lista delle adiacenze di w e quindi deve essere scoperto prima che w sia completato. Quindi si conclude che v è discendete di u. □

4.4.5 Tipi di archi DFS

Sia $uv \in E$ un arco, questo può essere classificato in:

- **Archi d'albero:** v è scoperto dalle adiacenze di u, quindi (v.status= UNEXPLORED)
- **Archi all'indietro:** u = v, oppure v è ascendente di u (v.status= EXPLORING)
- **Archi in avanti:** v discendete di u (v.status= EXPLORED AND v.timestamp > u.timestamp)
- **Archi trasversali:** u e v appartengono a rami diversi (v.status= EXPLORED AND v.timestamp < u.timestamp)

Il timestamp a cui fanno riferimento è quello di scoperta dei nodi.

Teorema

Un grafo orientato è aciclico (un DAG) se, e solo se, nella visita in profondità non si trova nessun arco all'indietro.

Dimostrazione:
⇒) Se in una visita in profondità si trova un arco all'indietro vu, allora tale arco aggiunto al cammino da u a v (che esiste in quanto v è discendente di u) forma un ciclo.

⇐) Viceversa, supponiamo che il grafo abbia un ciclo. Sia v il primo vertice del ciclo ad essere scoperto e sia uv l'arco del ciclo che entra in v. Quando v viene scoperto, esiste un cammino bianco da v ad u e quindi, per la proprietà del cammino bianco, u è discendente di v. Di conseguenza uv è un arco all'indietro. □

4.4.6 Ordinamento Topologico

Un Ordinamento Topologico di un DAG (Directed Acyclic Graph) G =(V, E) è un ordinamento totale di tutti i suoi vertici tale che, se G contiene un arco (u, v), allora u appare prima di v nell'ordinamento.

Topological-Sort(G)

1. Chiama DFS(G) per calcolare i tempi di completamento v.f per ciascun vertice v
2. Una volta completata la visita di un vertice, inserisce il vertice in testa a una lista concatenata
3. return la lista concatenata dei vertici

4.4.7 Componenti fortemente connesse

Una componente fortemente connessa (CFC) di un grafo orientato G = (V,E) è un insieme massimale di vertici $U \subseteq V$ tale che, $\forall\ u, v \in U$ esiste un cammino da u a v ed un cammino da v ad u.

Per trovare le componenti fortemente connesse di un grafo è sufficiente seguire questi passi:

1. Chiamare DFS sul grafo e calcolare i tempi di completamento per ciascun vertice;
2. Creare il grafo trasposto di G, ottenuto invertendo il verso di tutti gli archi;
3. Chiamare DFS sul grafo trasposto considerando i vertici in ordine decrescente di tempo di completamento calcolato al punto 1.;
4. Restituire o stampare le componenti fortemente connesse del grafo.

4.5 Alberi di connessione minimi MST

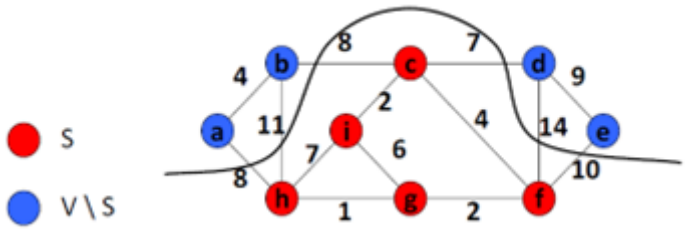
4.5.1 Il problema degli alberi di connessione minimi

Sia un grafo non orientato G=(V, E) connesso e i cui archi hanno associato un peso $\omega(e)$. Il problema di definire un albero di connessione minimo è quello di trovare un sottoinsieme T di archi tali che:

1. T rappresenta un albero
2. Il peso $\omega(T) = \sum_{e \in T} \omega(e)$ sia minimo

Un qualsiasi albero di connessione ha esattamente $|V| - 1$ archi.

4.5.2 Definizioni preliminari



- Un **taglio** in un grafo non orientato G=(V, E) è una partizione dell'insieme di vertici V in due sottoinsiemi non vuoti (S, V\S).
- Un arco **interseca** il taglio (S, V\S) se uno dei suoi estremi è in S e l'altro in V\S.
- Un taglio (S, V\S) **rispetta** un insieme di archi A se nessun arco in A interseca il taglio.
- Un arco **leggero** per il taglio è un arco di costo minimo tra tutti quelli che intersecano il taglio.

4.5.3 Algoritmo greedy per generare un albero di connessione minimo

Gli algoritmi avidi sono una particolare classe di algoritmi che risolvono i problemi di ottimizzazione adottando ad ogni passo la strategia migliore, ossia ragionano su un ottimo locale. Un metodo per generare un albero di connessione minimo da un grafo è il seguente:

```
GENERIC−MST(G){
    While "A non forma un albero di connessione"
        Trova un arco (u, v) che e' sicuro per A
        A= A U (u, v)
    return A
}
```

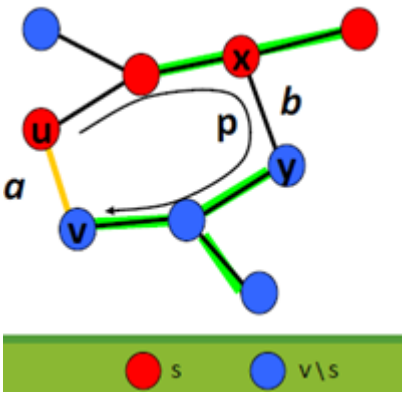
Questo metodo fa crescere l'albero di connessione un arco alla volta. Si nota come vengono aggiunti solo **archi sicuri** per A, ovvero quegli archi che consentono al metodo di conservare la seguente invariante di ciclo: **di ogni iterazione, A è un sottoinsieme di qualche albero di connessione minimo.**

4.5.4 Teorema: Caratterizzazione degli archi sicuri

Sia G=(V,E) un grafo connesso non orientato con una funzione Ω a valori reali definita in E. Sia A un sottoinsieme di E, che è contenuto in qualche albero di connessione minimo per G, sia (S, V\S) un taglio qualsiasi di G che rispetta A e sia **a= (u, v)** un arco leggero per il taglio (S, V\S). Allora l'arco a è sicuro.

Dimostrazione:
Sia T un albero di connessione minimo che contiene A. Se T contiene anche l'arco a, allora il teorema è dimostrato in quando a è un arco sicuro per A.

Sia quindi T l'albero di connessione minima in figura, se T non contiene l'arco a= (u, v), aggiungendo tale arco si genera un ciclo (banalmente in un albero non può esserci un ciclo).



Essendo T un albero, deve esistere necessariamente un percorso **p** che congiunga i vertici u e v. In aggiunta dato che u e v si trovano su lati opposti del taglio (S, V\S) c'è almeno un altro arco **b= (x, y)** tale che intersechi il taglio. L'arco b non appartiene ad A, in quanto come ipotesi del Teorema deve essere che il taglio (S, V\S) **rispetta** A.

Sia T' l'albero di connessione minimo ottenuto da T togliendo l'arco b e aggiungendo l'arco a. Dato che a come ipotesi è un arco leggero, si ha che il costo di T' è minore o uguale di T. Per definizione T è un albero di connessione minimo, ma allora anche T' lo è.

Infine essendo che T' contiene sia A che a, si ottiene che a rispetta A, in quanto aggiungendo questo arco ad A, questo è ancora un sottoinsieme di albero di connessione minimo. □

4.5.5 Algoritmo di Kruskal

L'algoritmo di Kruskal trova un arco sicuro da aggiungere all'albero di connessione scegliendo, fra tutti gli archi che collegano due alberi qualsiasi nella foresta, un arco (u, v) di peso minimo.

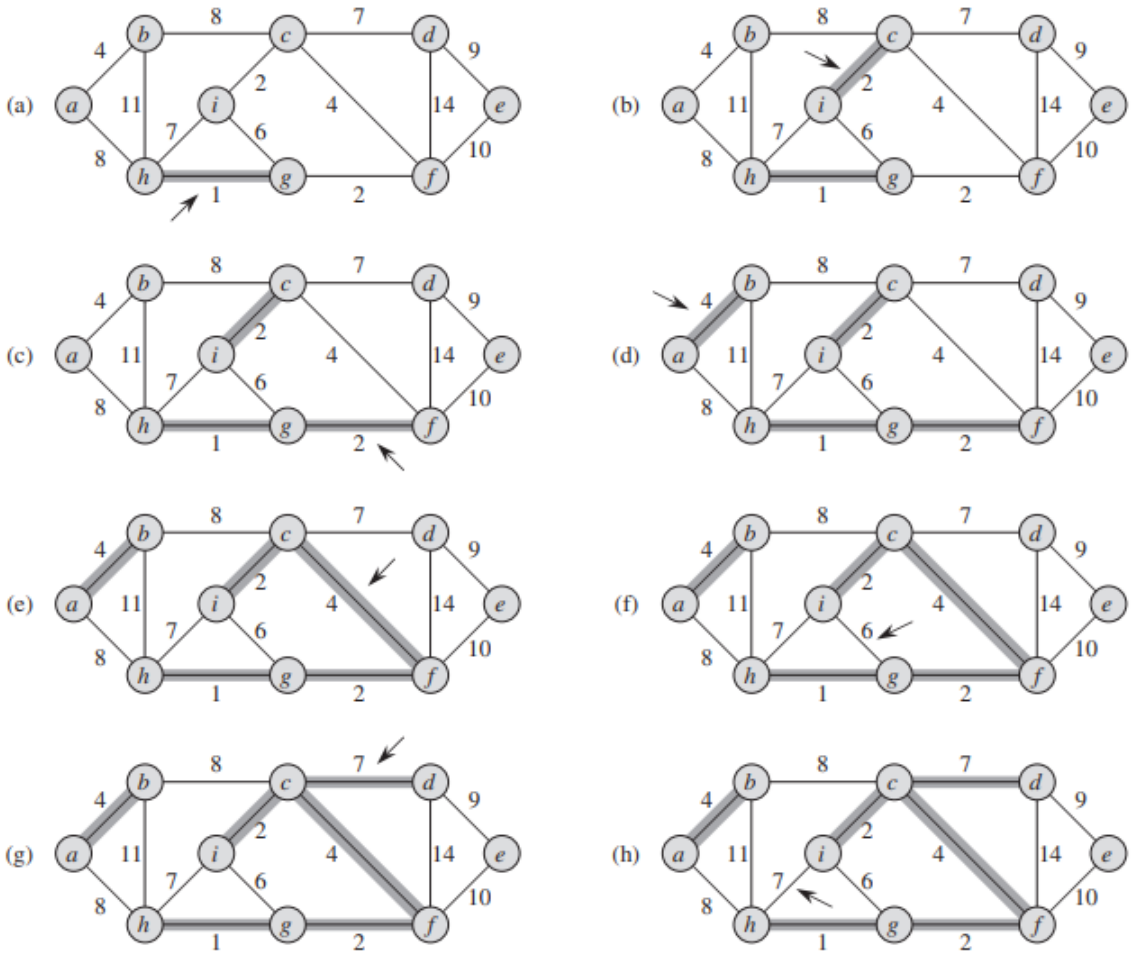
Pseudo Codice algoritmo di Kruskal

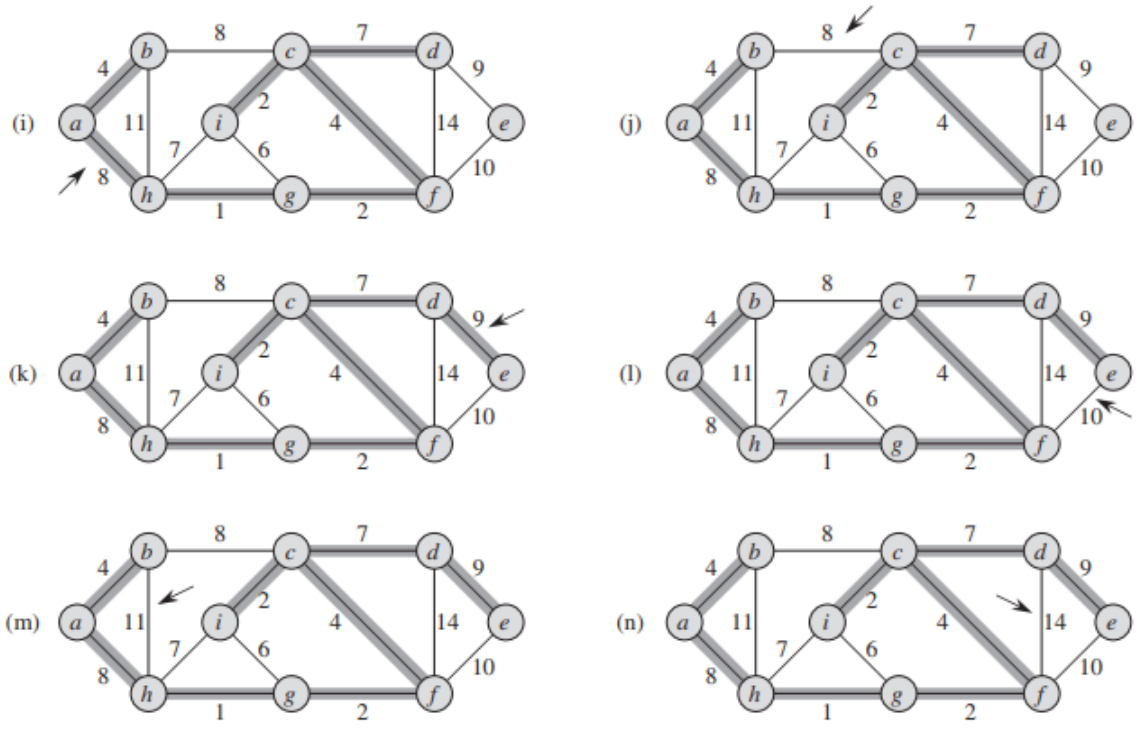
```
MST−KRUSKAL(G){
    A= 0
    foreach v in G.V
        Make−Set(v)

    l= converti_in_lista (G, E) // crea una lista di archi di G
    sort_decreasing (l, w) // ordina la lista in ordine monotono crescente rispetto al peso

    foreach (u, v) in l
        if Find−Set(u) != Find−Set(v)
            A= A U (u, v)
            Union(u, v)
    return A
}
```

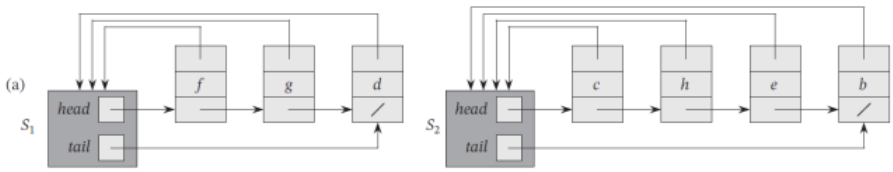
Esempio





Analisi

- $C = \{S_1, S_2\}$
  - S1={f, g, d} dove f è il rappresentante
  - S2={c, h, e, b} dove c è il rappresentante



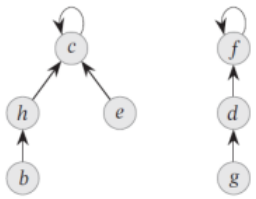
Per procedere con questo algoritmo è necessario definire una struttura dati che definisce una collezione di insiemi disgiunti (l'intersezione tra i vari insiemi è vuota), in cui ogni insieme è individuato da un rappresentante che è uno degli elementi dell'insieme. In questo algoritmo si è fatto uso di tre funzioni principali:

- **Make-Set(x):** aggiunge alla struttura dati un nuovo insieme contenente solo l'elemento x (si richiede che x non compaia in nessun altro insieme della struttura). Ha costo **O(1)**.
- **Find-Set(x):** restituisce il rappresentate dell'insieme che contiene x. Ha costo **O(1)**.
- **Union(x, y):** unisce i due insiemi contenti x ed y. Ha costo che dipende dalla dimensione di y e tende ad n.

Il costo dell'algoritmo di Kruskal è $O(n^2)$, il costo dipende principalmente dall'andamento della funzione **Union**. Quindi per migliorare il costo di questo algoritmo è necessario modificare l'implementazione della funzione.
Questo algoritmo può essere implementato con un'altra struttura dati per rappresentare gli insiemi disgiunti: **le foreste di insiemi disgiunti**, in cui ogni insieme è rappresentato da un albero i cui nodi, oltre al campo data che contiene l'informazione, hanno un campo **p** che punta al predecessore.

$C = \{S_1, S_2\}$

- S1={c, h, e, b} dove c è il rappresentante
- S2={f, d, g} dove f è il rappresentante



In questo caso possiamo implementare le tre funzioni in questo modo:

```
Make−Set(x){
    x.p= x
}
Find−Set(x){
    if x.p != x
        Find−Set(x.p)
    return x.p
}
Union(x, y){
    x= Find−Set(x)
    y= Find−Set(y)
    x.p= y
}
```

Si può notare che il costo della **Union** è uguale al costo della **Find-Set**, ovvero $O(log\ n)$, in quanto devo scorrere da x alla radice dell'albero. Anche in questo caso il costo dell'algoritmo di Kruskal implementando quest'altro tipo di struttura dati è $\mathbf{O(n^2)}$.

4.5.6 L'algoritmo di Prim

A differenza dell'algoritmo di Kruskal che iterava sul costo degli archi in ordine crescente, l'algoritmo di Prim, per generare un albero di connessione minima, ragiona con una coda di min proprietà (come può essere ad esempio un min-Heap) le cui chiavi dei nodi indicano il peso minimo di un arco qualsiasi che collega il nodo ad un altro vertice nel grafo.

Pseudo Codice algoritmo di Prim

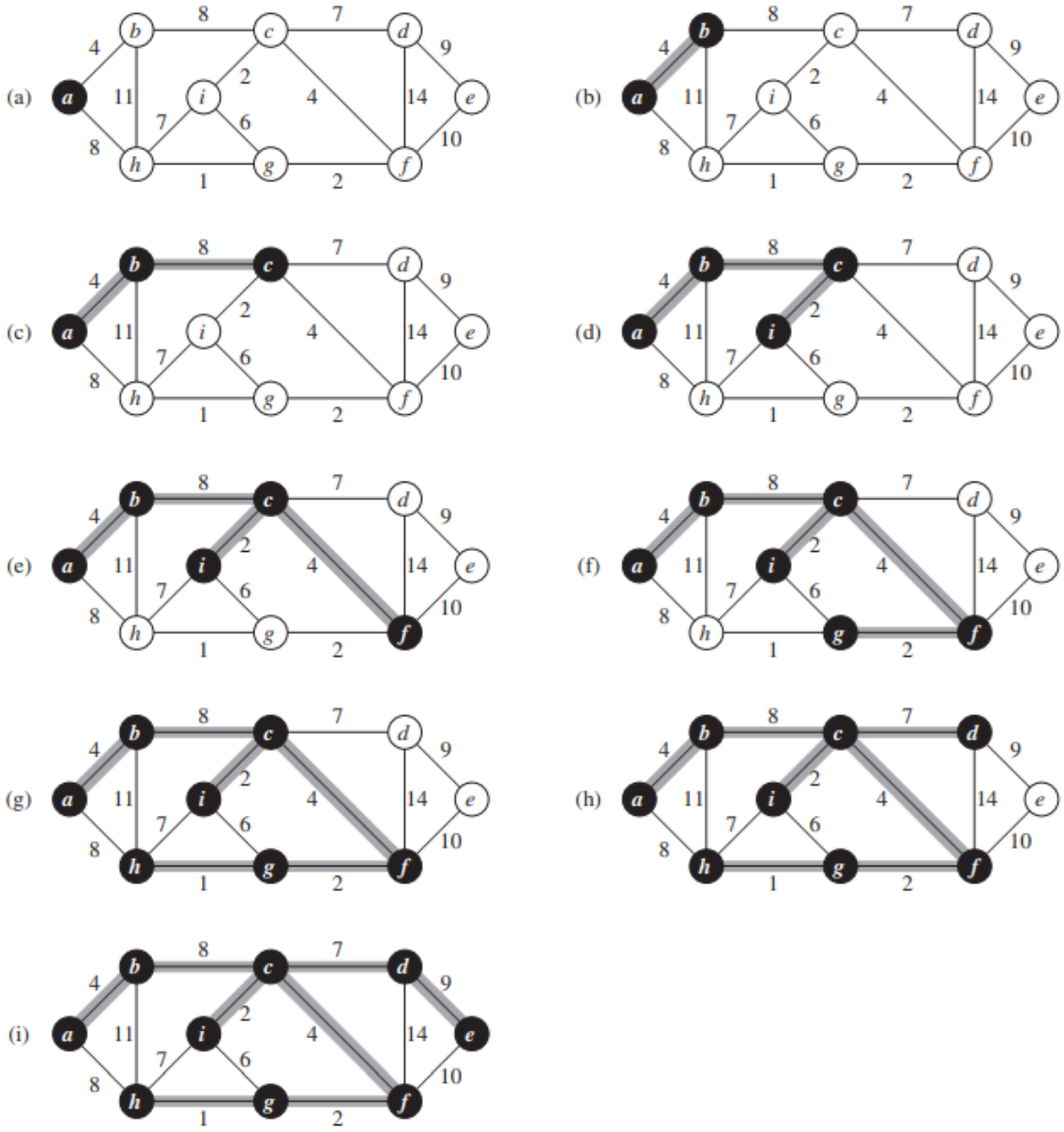
```
MST−Prim(G, s){
    foreach u in G.V
        u.key= infinity
        u.p= nil
    s.key= 0
    Q= 0

    foreach u in G.V
        Insert(Q, u)

    while Q != 0
        u= Extract_Min(Q)
```

```
        foreach v in G.Adj[u]
            if v in Q and w(u, v) < v.key
                v.p= u
                v.key= w(v, u)
                Decrease_key(Q, v, w(u, v))
}
```

Esempio



Innanzitutto si sceglie un nodo da cui partire che fungerà da sorgente, in questo caso il nodo a. Successivamente l'algoritmo inizializza tutte le chiavi dei nodi ad infinito e i predecessori a nil, mentre per quanto riguarda la sorgente pone la chiave a 0 e aggiunge tutti i nodi al min-heap. Quindi la coda adesso è formata da:

$$Q = [(a, 0), (b, \infty), (c, \infty), (d, \infty), (e, \infty), (f, \infty), (g, \infty), (h, \infty), (i, \infty)]$$

Successivamente l'esecuzione entra nel while, che scansione l'intera coda svuotando man mano il min-heap. L'algoritmo prende il primo nodo (in questo caso il vertice a) e scansiona la sua

lista di adiacenza (contenente i vertici b ed h). Si valuta quindi se il costo dell'arco (prima (a, b) e poi (a, h)) è minore della chiave del nodo e successivamente se la condizione è verificata va ad aggiornare la chiave del vertice nel min-heap. Quindi la coda adesso diventa:

$$Q = [(b, 4), (h, 8), (c, \infty), (d, \infty), (e, \infty), (f, \infty), (g, \infty), (i, \infty)]$$

Si procede con il nodo b e vengono valutati gli archi (b, c), (b, h), aggiornando il valore della chiave del nodo c, ma non quello del nodo h in quanto $h.key < \omega(b.h)$, quindi la condizione non è verificata. In questo caso nel min-heap, dato che c ed h hanno lo stesso valore di chiave, si può decidere se visitare prima il nodo c o prima il nodo h, in entrambi i casi avremo un albero di connessione a costo minimo.

$$Q = [(c, 8), (h, 8), (d, \infty), (e, \infty), (f, \infty), (g, \infty), (i, \infty)]$$

Si procede in seguito con il nodo c e si valutano gli archi (c, i), (c, f), (c, d) e la coda diventa:

$$Q = [(i, 2), (f, 4), (d, 7), (h, 8), (e, \infty), (g, \infty)]$$

Dopodiché si procede con il vertice i e vengono valutati gli archi (i, h), (i, g) e la coda diventa:

$$Q = [(f, 4), (g, 6), (d, 7), (h, 7), (e, \infty)]$$

Dopo è il turno del nodo f e vengono valutati gli archi (f, g), (f, d), (f, e), la coda diventa:

$$Q = [(g, 2), (d, 7), (h, 7), (e, 10)]$$

Si estrae il nodo g e viene valutato l'arco (g, h):

$$Q = [(h, 1), (d, 7), (e, 10)]$$

Si procede con il nodo h e non vengono valutati archi in quanto nessun vertice a cui è connesso h è nella coda:

$$Q = [(d, 7), (e, 10)]$$

Si rimuove d dalla coda e viene valutato l'arco (d, e):

$$Q = [(e, 9)]$$

Infine, non ci sono più archi da valutare e si svuota la coda.

4.6 Cammini minimi da sorgente unica

Sia G = (V, E) un grafo con funzione di peso $w : E \to \mathbb{R}$ che assegna ad ogni arco un peso di valore reale. Il problema dei cammini minimi sul grafo G a partire da una sorgente s, con s in V, consiste nel trovare un percorso di peso minimo che congiunga s con qualsiasi nodo v in V.

In un problema dei cammini minimi è dato un grafo orientato pesato G = (V, E), con una funzione peso ω che associa agli archi dei pesi di valore reale. Il peso $\omega(p)$ del cammino $p = (v_0, v_1, ..., v_k)$ è la somma dei pesi degli archi che lo compongono:

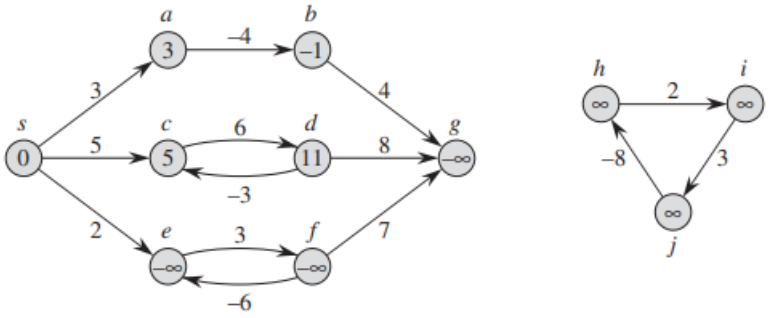
$$\omega(p) = \sum_{i=1}^{k} \omega(v_{i-1}, v_i)$$

Il peso di cammino minimo da un vertice u ad un vertice v è definito nel seguente modo:

$$\delta(u, v) = \begin{cases} min\{\omega(p)\} & \text{se esiste un cammino p da u a v} \\ \infty & \text{altrimenti} \end{cases}$$

Un **cammino minimo** da u a v è un cammino p da u a v di costo $\omega(p) = \delta(u, v)$.

4.6.1 Archi di peso negativi



In alcuni casi del problema dei cammini minimi da sorgente unica possono presentarsi degli archi i cui pesi sono negativi. Se il grafo G = (V, E) non contiene cicli di peso negativo che sono raggiungibili dalla sorgente s, allora per ogni vertice v, il peso del cammino minimo $\delta(s, v)$ resta ben definito, anche se ha un valore negativo. Tuttavia, se esiste un ciclo di peso negativo che è raggiungibile da s, i pesi dei cammini minimi non sono ben definiti. Nessun cammino da s a un vertice del ciclo può essere un cammino minimo in quanto è sempre possibile trovare un cammino di peso minore. Se esiste un ciclo di peso negativo in qualche cammino da s a v, si definisce $\delta(s, v) = -\infty$.

4.6.2 Proprietà dei cammini minimi

Sotto-struttura ottima dei cammini minimi
Se il cammino $p = (v_0, v_1, ..., v_k)$ è minimo, allora sono minimi anche tutti i sottocammini $p_{ij} = (v_i, ..., v_j)$.

Dimostrazione.
Se esistesse un cammino **q** da v_i a v_j di costo minore di p_{ij}, allora sostituendo nel cammino p il sottocammino p_{ij} con il cammino q si otterrebbe un cammino da v_0 a v_k di costo minore di p. Il che è impossibile se p è un cammino minimo. □

Scomposizione dei costi di cammino minimo
Se p è un cammino minimo da s ad un vertice v diverso da s ed u è il vertice che precede v nel cammino, allora:

$$\delta(s, v) = \delta(s, u) + \omega(u, v)$$

Dimostrazione.
Come conseguenza della sottostruttura ottima si ha che:

$$d(s, v) = \omega(p) = \delta(s, u) + \omega(u, v)$$

□

Limite superiore per i costi di cammino minimo
Per ogni arco uv vale la diseguaglianza:

$$\delta(s, v) \leq \delta(s, u) + \omega(u, v)$$

Dimostrazione.
Se u non è raggiungibile da s allora $d(s, u) = \infty$ e $\delta(s, v) \leq \infty + \omega(u, v)$. Se u è raggiungibile da s allora $\delta(s, u) + \omega(u, v)$ è il costo di un cammino da s a v ed è quindi maggiore o uguale di $\delta(s, v)$. □

4.6.3 Tecnica del rilassamento

Per ogni vertice v, manteniamo un attributo v.d, che è un limite superiore per il peso di un cammino minimo dalla sorgente s a v $\delta(s, v)$. L'attributo v.d è detto **stima del cammino minimo**, che alla fine dell'esecuzione sarà uguale a $\delta(s, v)$. Il processo di **rilassamento** di un arco (u, v) consiste nel verificare se, passando per u, è possibile migliorare il cammino minimo per v precedentemente trovato e, in caso affermativo, nell'aggiornare v.d.

Pseudo Codice

```
RELAX(u, v, w){
    if v.d > u.d + w(u, v)
        v.d= u.d + w(u, v)
}
```

Proprietà del rilassamento
Dopo l'inizializzazione, per ogni vertice v vale la disuguaglianza $\mathbf{v.d} \geq \delta(\mathbf{s}, \mathbf{v})$ che rimane vera anche dopo un numero qualsiasi di rilassamenti. Inoltre, se a un certo punto $\mathbf{v.d} = \delta(\mathbf{s}, \mathbf{v})$, allora il suo valore non può più cambiare.

Dimostrazione.
Dopo l'inizializzazione si ha $s.d = 0 \geq \delta(s, s)$ e per ogni altro vertice $v.d = \infty \geq d(s, v)$.

Supponiamo che la disuguaglianza sia vera per ogni vertice prima di eseguire **Relax(u, v, w)**. Se v.d non viene modificata, la disuguaglianza resta ovviamente vera. Se v.d viene modificata, allora dopo aver eseguito il rilassamento si ha $v.d = u.d + \omega(u, v)$. Poiché u.d non è stata modificata, vale la disuguaglianza $u.d \geq \delta(s, u)$, e quindi

$$v.d \geq \delta(s, u) + \omega(u, v)$$

Per il limite superiore dei costi di cammino minimo è nota la disequazione

$$\delta(s, v) \leq \delta(s, u) + \omega(u, v)$$

da cui deriva che $v.d \geq .\delta(s, v)$. □

4.6.4 Algoritmo di Bellman-Ford

L'algoritmo di Bellman-Ford risolve il problema dei cammini minimi da sorgente unica nel caso generale in cui i pesi degli archi possono essere negativi. Dato un grafo orientato pesato G = (V, E) con sorgente s e funzione peso ω, l'algoritmo di Bellman-Ford restituisce un valore booleano che indica se esiste oppure no un ciclo di peso negativo che sia raggiungibile dalla sorgente. Se un tale ciclo esiste, l'algoritmo indica che il problema non ha soluzione. Se un tale ciclo non esiste, l'algoritmo fornisce i cammini minimi e i loro pesi. L'algoritmo usa la tecnica del rilassamento, riducendo progressivamente il valore stimato v.d per il peso di un cammino minimo dalla sorgente s a ciascun vertice, fino a raggiungere il peso effettivo $\delta(s, v)$ di un cammino minimo. L'algoritmo restituisce **TRUE** se e soltanto se il grafo non contiene cicli di peso negativo che sono raggiungibili dalla sorgente.

Pseudo Codice

```
RELAX(u, v, w){
    if v.d > u.d + w(u, v)
        v.d= u.d + w(u, v)
}
```

```
INITIALIZE−SINGLE−SOURCE(G, s){
    for ogni vertice v in G.V
        v.d= infinity
        v.p= nil // non necessario
    s.d= 0
}
BELLMAN−FORD(G, s, w){
    INITIALIZE−SINGLE−SOURCE(G, s)
    for i=1 to |G.V|−1
        for ogni arco (u, v) in G.E
            RELAX(u, v, w)

    for ogni arco (u, v) in G.E
        if v.d > u.d + w(u, v)
            return FALSE
    return TRUE
}
```

4.6.5 Algoritmo di Dijkstra

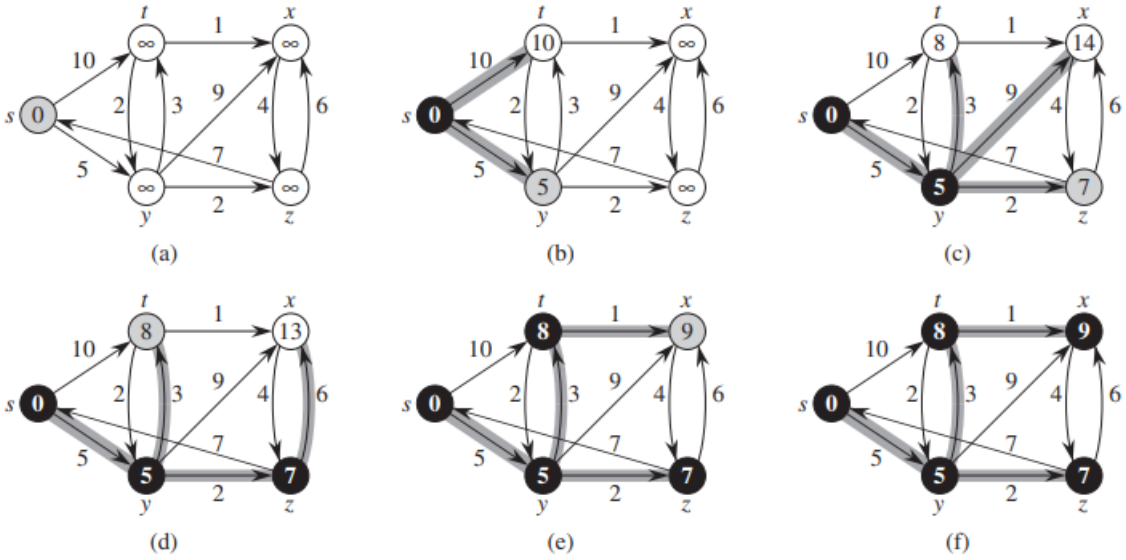
L'algoritmo di Dijkstra risolve il problema dei cammini minimi da sorgente unica in un grafo orientato pesato G = (V, E) nel caso in cui tutti i pesi degli archi non siano negativi. L'algoritmo di Dijkstra mantiene in una coda di min proprietà, come può essere un min-heap, le stime delle distanze di ogni vertice, esclusa la sorgente. L'algoritmo seleziona ripetutamente il vertice u nel min-heap con la stima minima del cammino minimo e applica la tecnica del rilassamento a tutti gli archi uscenti da u.

Pseudo Codice

```
Dijkstra(G, s){
    s.d= 0     // inizializzazione distanza sorgente
    s.p= NULL    // inizializzazione padre sorgente
    H= new empty min−heap   // l'heap una volta riempito avra' dimensione |V|
    for v in G.V
        if(v != s)      // tutti i vertici diversi dalla sorgente vengono inizializzati ad infinito
            v.d= INFINITY
            v.p= UNDEFINED
        H.add(v, v.d)

    while(!H.isEmpty())     // si va avanti fino a svuotare tutto l'heap
        u= H.extractMin()
        for v in G.Adj[u]   // per qualunque vicino di u
            // w(u, v) peso dell'arco
            if(v.d > u.d + w(u, v))   // percorso migliore di v dalla sorgente
                v.d= u.d + w(u, v))
                // alla distanza di v dalla sorgente viene assegnata la distanza migliore
                v.p= u
                H.decreasePriority(v, v.d)  // modifica la chiave di v
}
```

Esempio



Al punto a) tutti i vertici sono stati inizializzati, successivamente viene estratta la sorgente e viene applicato il rilassamento sui nodi t e y, rispettivamente gli vengono assegnati le chiavi 10 e 5. All'iterazione successiva, viene estratto il nodo y (essendo la chiave più piccola) e vengono rilassati gli archi (y, t), (y, x) e (y, z). Al punto d) è stato estratto il nodo z e si applica il rilassamento all'arco (z, x). In seguito viene estratto il nodo t e viene rilassato l'arco (t, x). Infine anche il nodo x viene estratto, dato che non ha archi uscenti e il min-heap è stato svuotato, l'algortmo termina l'esecuzione.

5 Gli alberi

Un albero è un grafo connesso senza cicli, ovvero un DAG connesso. Una delle caratteristiche fondamentali degli alberi è che essi hanno esattamente n-1 archi, proprietà che può essere proposta anche per un grafo connesso senza cicli ricordando che il numero n di nodi che costituiscono un albero può essere inteso come $|V|$, dove V è l'insieme dei vertici del grafo G. Dunque per verificare che un grafo non abbia cicli è sufficiente stabilire che abbia un numero di archi pari a n-1. In questo caso, essendo fornito un grafo semplice e connesso con n nodi ed m archi basta implementare un algoritmo che verificare che m = n-1, ovvero effettuare un confronto semplice con costo costante O(1).

5.1 Tipi di albero

Albero libero: Un albero libero è un tipo di grafo non orientato connesso e senza cicli, ovvero un DAG (Directed Acyclic Graph) connesso. Non è caratterizzato da uno specifico nodo che assume il ruolo di radice, bensì qualsiasi nodo può esserlo.

Albero radicato: Un albero radicato è una tipologia di albero libero in cui un vertice è stato scelto come radice. La radice è il punto di partenza da cui si può raggiungere ogni altro nodo attraverso un percorso unico.

Albero ordinato: Un albero radicato si dice **ordinato** se i figli di ogni nodo hanno un ordine specifico. Inoltre l'ordine può essere necessario per la visita pre-ordine, in-ordine e post-ordine.

Albero posizionale: Un albero ordinato si dice posizionale se ad ogni figlio di un nodo è associata una posizione specifica, ad esempio quando si parla di figlio "destro" e "sinistro" per gli alberi binari. Ricorda: le posizioni sono etichettate e differenziano i figli di un nodo. Quando non sono occupate da un nodo sono posizioni vuote.

Albero k-ario: Un albero k-ario è un albero posizionale in cui ogni posizione maggiore di k è vuota, ciò si traduce nel dire che ogni nodo ha al massimo k figli. Viene utilizzato per rappresentare strutture con una quantità fissa di sottoelementi.

Albero binario: Un albero binario è un esempio di albero k-ario in cui ogni nodo ha al massimo due figli, pertanto k = 2. I due figli sono tipicamente chiamati "destro" e "sinistro". Questa tipologia di albero è fondamentale per molte strutture dati e algoritmi, come gli alberi binari di ricerca e gli heap.

5.2 Gli alberi binari

5.2.1 Rappresentazione degli alberi binari

Ogni nodo dell'albero è composto dai seguenti attributi

1. Puntatore al nodo padre: **p**;
2. Puntatore al figlio sinistro: **l**;
3. Puntatore al figlio destro: **r**;
4. Chiave del nodo: **key**, ovvero l'attributo in cui viene memorizzata l'etichetta del nodo.

5.2.2 Albero binario di ricerca BST

Un albero binario di ricerca, definito con la sigla BST, è un albero binario che soddisfa la seguente proprietà:

Sia x un nodo di un albero binario di ricerca, se y è un nodo del sottoalbero sinistro di x allora $y.key \leq x.key$*, mentre se y è un nodo del sottoalbero destro di x allora* $y.key \geq x.key$*.*

5.2.3 Visita di un albero binario

Diversamente dal caso delle strutture lineari, dove è possibile visitare tutti gli elementi in sequenza nell'ordine in cui sono rappresentati, nel caso di alberi binari vi sono modi diversi di esplorare i nodi. In particolare, le tre operazioni caratteristiche della visita in profondità (visita della radice, visita del sottoalbero sinistro, visita del sottoalbero destro) possono essere realizzate in qualunque ordine. Esistono tre versioni di visita in profondità:

1. **Attraversamento simmetrico o visita in ordine (inorder tree walk):** sottoalbero sinistro, radice, sottoalbero destro. Ciò implica che i nodi vengono visitati in ordine crescente di chaiave.

2. **Attraversamento anticipato (preorder tree walk):** radice, sottoalbero sinistro, sottoalbero destro. Ciò implica che la radice venga visitata prima dei suoi sottoalberi.

3. **Attraversamento posticipato (postorder tree walk)**: sottoalbero sinistro, sottoalbero destro, radice. Ciò implica che la radice venga visitata dopo dei suoi sottoalberi.

Pseudo Codice funzioni per visita di un albero binario

```
Pre−order(x){
    if x != nil
        print x.key
        Preorder(x. left )
        Preorder(x.right )
}
Post−order(x){
    if x != nil
        Postorder(x. left )
        Postorder(x.right )
        print x.key
}
In−order(x){
    if x != nil
        Inorder(x. left )
        print x.key
        Inorder(x. right )
}
```

5.2.4 Analisi della complessità

Teorema. Se x è la radice di un sottoalbero con n nodi allora **Inorder(x)** richiede un tempo $\boldsymbol{\Theta}(\mathbf{n})$.

Dimostrazione:
Sia T(n) il tempo richiesto da Inorder(x) per visitare un sottoalbero di n nodi con radice x, è

necessario dimostrare che T(n) è $\Omega(n)$ e allo stesso tempo $O(n)$.

Dire che T(n) è $\mathbf{\Omega(n)}$ significa che T(n) è almeno pari a $\mathbf{c \cdot n}$ per qualche costante c e per n sufficientemente grande. Durante l'attraversamento in-ordine dobbiamo visitare tutti i nodi, ogni nodo deve essere visitato almeno una volta, ecco spiegato il motivo per il quale il tempo richiesto per visitare tutti i nodi è almeno proporzionale ad n.

Dire che T(n) è $\mathbf{O(n)}$ significa che T(n) è al massimo $\mathbf{c \cdot n}$ per qualche costante c e per n sufficientemente grande. Sapendo che l'algoritmo di attraversamento in-ordine segue questi passi:

1. Visita il sottoalbero sinistro;
2. Visita la radice x;
3. Visita il sottoalbero destro;

Supponiamo che $\mathbf{n = 0}$, per visitare un nodo serve una piccola quantità di tempo, basta verificare che x = nil, ecco perché $T(n = 0) = c$.
Supponendo che n > **0** e che il sottoalbero sinistro abbia k nodi e il sottoalbero destro ne abbia n-k-1, il tempo totale T(n) è quindi minore o uguale alla somma dei tempi richiesti per visitare il sottoalbero sinistro, la radice e il destro:

$$T(n) \leq T(k) + T(n–k–1) + d$$

dove d è il tempo costante per visitare la radice x (che è un'operazione O(1)). Applichiamo il metodo di sostituzione per dimostrare che:

$$\mathbf{T(n) = (c + d)n + c} \text{ per ogni } \mathbf{n \geq 0}$$

.
Caso base (n = 0): $T(0) = c = (c + d) \cdot 0 + c$

Per n > 0:

$T(n) \leq T(k) + T(n–k–1) + d$
$= ((c + d)k + c) + ((c + d)(n–k–1) + c) + d$
$= (c + d)k + c + (c + d)n–(c + d)k–(c + d) + c + d$
$= c–c–d + d + c + (c + d)n$
$= (c + d)n + c$ □

5.2.5 Interrogazioni di un albero binario di ricerca

I BST sono strutture dati sulle quali vengono realizzate molte delle operazioni definite su insiemi dinamici, anche chiamate "interrogazioni".

1. **Search(x, key):** data la radice x e una chiave key, restituisce il puntatore a un nodo con chiave key, se esiste, o nil;
2. **Minimum(x) o Maximum(x):** dato un albero con radice x, restituisce l'elemento la cui chiave è minima o massima;
3. **Successor(x) o Predecessor(x):** dato un albero con radice x, restituisce il nodo successivo o precedente a x rispetto all'attraversamento simmetrico.

Pseudo Codice

```
Search(x, k){
    if x == nil or k == x.key
        return x
    if k < x.key
        return Search(x.leZ, k)
    else
        return Search(x.right, k)
}
Minimum(x){
    while x.left != nil
        x = x.left
    return x
}
Maximum(x){
    while x.right 1 nil
        x = x.right
    return x
}
Successor(x){
    if x.right != nil
        return Minimum(x.right)
    else
        y = x.p
        while y != nil and x == y.right
            x = y
            y = x.p
        return y
}
Predecessor(x){
    if x.left != nil
        return Maximum(x.left)
    else
        y = x.p
        while y != nil and x == y.left
            x = y
            y = x.p
        return y
}
```

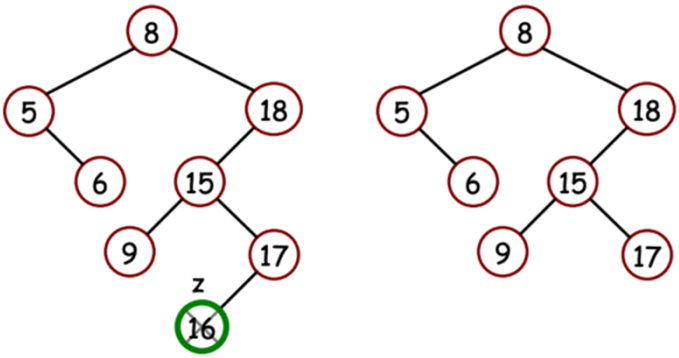
Tutte queste funzioni hanno costo $\mathbf{O(h)}$, dove h è l'altezza del nodo x nell'albero.

Tra queste vi è una categoria "a parte" che riguarda le operazioni che agendo sull'albero ne modificano il contenuto. Nel caso di alberi binari di ricerca queste operazioni devono essere realizzate preservando la proprietà dell'ordinamento delle chiavi. Ciò significa che è necessario verificare se un inserimento o cancellazione interferiscono con tale proprietà, in tal caso è necessario aggiornare l'albero affinchè sia possibile ripristinarla.

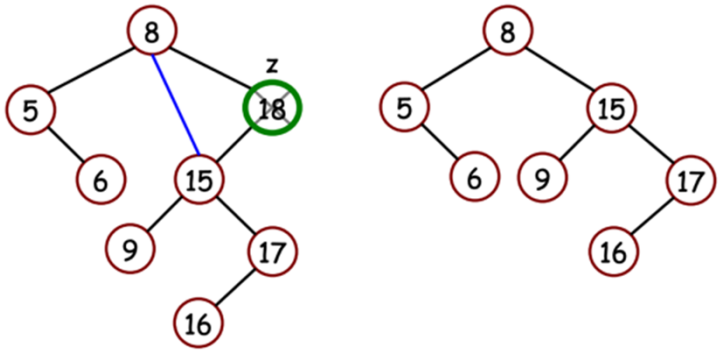
1. **Insert(T, z):** l'idea è quella di inserire nell'albero un nuovo nodo z sottoforma di foglia. Affinché ciò sia possibile, è necessario trovare una foglia che possa diventare per essa il padre senza compromettere le proprietà dell'albero;

2. **Delete(T, z):** il ragionamento per questo tipo di operazione non è unico, bensì ci si può trovare in tre situazioni diverse:

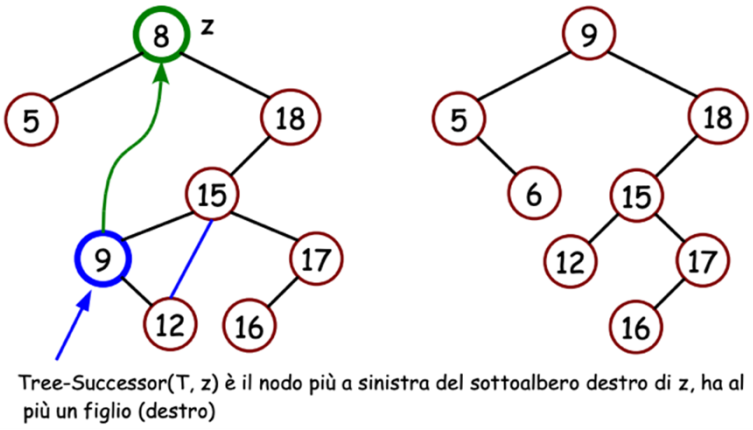
   - **z è una foglia**, ovvero non ha figli: si elimina il nodo modificando a nil il puntatore del padre;



   - **z ha un solo figlio, w:** in tal caso è come se il nodo w venisse "promosso" a prendere il posto di z nell'albero, aggiornando il puntatore del padre di z in modo che punti a w;



   - **z ha due figli:** bisogna trovare il successore y di z nel sottoalbero destro, spostare y nella posizione precedentemente coperta da z, infine ricollegare i sottoalberi. Per spostare y bisogna tener conto di una condizione, ovvero se y è figlio destro di z semplicemente lo si fa risalire per poi "attaccargli" il vecchio sottoalbero sinistro di z, altrimenti si scambia y con il suo figlio destro per poi portare verso l'alto y fino a sostituire z.



Pseudo Codice

```
Insert(T, z){
    x = T.root
    y = nil
    while x !=NULL // trovo il nodo dove attaccare z
        y = x
        if z.key < y.key
            x = y.left
        else
            x = y.right
    z.p = y

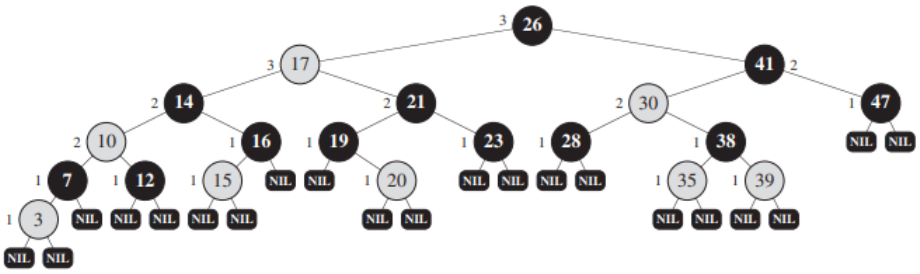
    if y == nil // l'albero e' vuoto
        T.root = z // z diventa la radice
    else if z.key < y.key // inserisco z nell'albero
        y.left = z
    else
        y.right = z
}
Transplant(T, u, v){
     if u.p == nil
        T.root = v
    else if u == u.p.left
        u.p.left = v
    else
        u.p.right = v
    if v != nil
        v.p = u.p
}
Delete(T, z){
    if z.left == nil
        // se il sottoalbero sinisto di z e' vuoto, devo attaccare z.right a z.p
        Transplant(T, z, z.right)
    else if z.right == nil
        // se il sottoalbero destro di z e' vuoto, devo attaccare z.left a z.p
        Transplant(T, z, z.left)
    // cosi' facendo ho trattato i casi se z ha un solo figlio oppure e' una foglia
    else
        y = Minimum(z.right) // trovo y, il successore di z
        // sposto y nella posizione di z, tenendo conto dei sottoalberi di y
        if y != z.right
            Transplant(T, y, y.right)
            y.right = z.right
            y.right.p = y
        Transplant(T, z, y)
        y.left = z.left
        y.left.p = y
}
```

5.3 Alberi Rosso-Neri

Un albero rosso-nero è un albero binario di ricerca con un bit in più di memoria per ogni nodo necessario per il colore, che può essere RED o BLACK. L'albero si dice **approssimativamente bilanciato** poiché, grazie a specifici vincoli per assegnare i colori ai nodi, garantisce che nessun cammino che va dalla radice a una foglia sia lungo più del doppio di qualsiasi altro.
Ogni nodo ha i seguenti attributi: color, key, left, right e p. Se manca un figlio o il padre di un nodo, il corrispondente attributo puntatore del nodo contiene il valore NIL, il quale viene trattato come un puntatore a una foglia.

5.3.1 Proprietà di un albero rosso-nero

1. Ogni nodo è rosso o nero;
2. La radice è nera;
3. Ogni foglia (NIL) è nera;
4. Se un nodo è rosso, allora entrambi i suoi figli sono neri;
5. Per ogni nodo, tutti i cammini semplici che vanno dal nodo alle foglie sue discendenti contengono lo stesso numero di nodi neri.
6. L'altezza massima di un albero rosso-nero con n nodi interni è $2 \cdot log(n+1)$.



Per un albero rosso-nero si definisce sentinella, l'oggetto T.nil con gli stessi attributi di un nodo ordinario dell'albero, il suo colore è "BLACK" e gli attributi p, left, right e key possono assumere valori arbitrari. Bisogna notare che tutti i puntatori a NIL sono sostituiti con puntatori alla sentinella. L'uso di quest'ultima permette di trattare un figlio NIL di un nodo x come un nodo ordinario il cui padre è x.
Si definisce altezza nera di un nodo x, indicata da **bh(x)**, il numero di nodi neri lungo un cammino semplice che inizia dal nodo x (ma non lo include) e finisce in una foglia. La proprietà 5 permette di definire questo concetto, infatti tutti i cammini semplici che scendono dal nodo hanno lo stesso numero di nodi neri. Per definizione, l'altezza nera di un albero rosso-nero è l'altezza nera della sua radice.

6 Hashing

I dizionari sono strutture dati fondamentali che consentono di rappresentare una relazione tra un insieme di chiavi D, il dominio, e un insieme di valori C, il codominio. Questa associazione chiave-valore è comunemente utilizzata in diverse applicazioni come le rubriche telefoniche, dove ogni numero di telefono (chiave) è associato a un nome (valore). Le operazioni principali che devono essere supportate efficientemente da un dizionario sono:

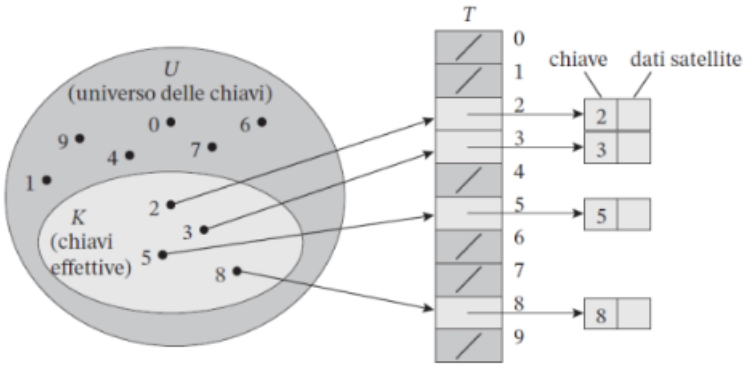
- **Insert:** aggiunge un nuovo valore;
- **Search:** restituisce il valore associato a una chiave;
- **Delete:** elimina un valore dalla struttura.

Esistono diversi modi per implementare un dizionario, tra cui:

- Tabelle ad indirizzamento diretto;
- Tabelle hash.

6.1 Tabelle ad indirizzamento diretto

L'indirizzamento diretto è una tecnica efficace quando **U**, l'universo delle chiavi, è ragionevolmente piccolo, ovvero dato $U = \{0,\ 1,\ ...,\ m-1\}$, con m non troppo grande. Le tabelle ad indirizzamento diretto utilizzano un array per gestire le chiavi e rappresentare l'insieme dinamico in cui ogni elemento ha associata una chiave estratta dall'universo U. Supponendo che due elementi non possano avere la stessa chiave, si può rappresentare l'insieme dinamico utilizzando un array o **tavola a indirizzamento diretto** indicata con $T[0\ ...\ m-1]$, dove ogni posizione o cella dell'array è associata a una chiave specifica, e il valore associato ad essa è memorizzato nell'array. La cella k punta a un elemento dell'insieme con chiave k. Se nessun elemento è associato ad una specifica chiave k, allora $T[k] = NIL$.



6.1.1 Operazioni su una tabella ad indirizzamento diretto

Le operazioni su una tabella ad indirizzamento diretto sono le seguenti:

- **Search(T, k):** restituisce T[k];
- **Insert(T, x):** imposta T[x.key] = x;
- **Delete(T, x):** imposta T[x.key] = nil.

Analisi

I principali **vantaggi** delle tabelle a indirizzamento diretto includono operazioni di inserimento, ricerca e cancellazione molto efficienti in tempo $\mathbf{O(1)}$.

Bisogna notare quanto sia vantaggiosa la possibilità di evitare di memorizzare la chiave e i dati satelliti di un elemento in un oggetto esterno puntato da una cella dell'array T, al contrario è possibile memorizzare l'oggetto nella cella stessa, risparmiando spazio e usando l'indice come chiave.

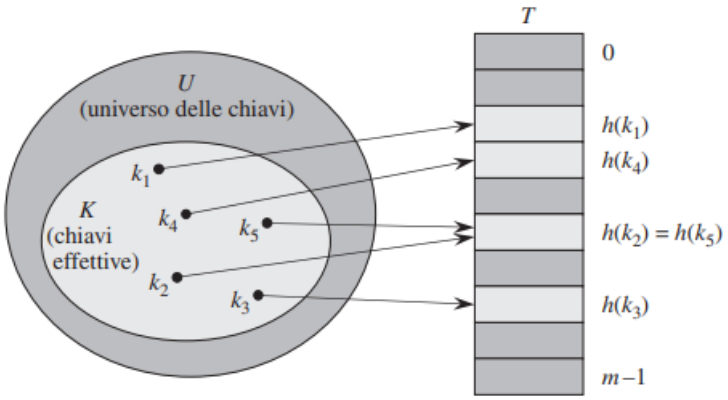
Tuttavia, presentano anche significativi **svantaggi**, infatti con l'indirizzamento diretto si è "costretti" ad allocare memoria per una tavola T di dimensione $|\mathbf{U}|$, ovvero pari alla cardinalità dell'insieme dell'universo delle chiavi. Se l'universo è troppo grande può essere impraticabile allocare così tanta memoria, inoltre potrebbe succedere che l'insieme K delle chiavi effettivamente memorizzate sia così piccolo rispetto a U che la maggior parte dello spazio allocato per la tavola T sarebbe sprecato.

6.1.2 Fattore di carico

Il fattore di carica α misura il grado di riempimento di una tabella ed è definito come $\alpha = n/m$ dove $n = |K|$ è il numero di chiavi effettivamente utilizzate e $m = |U|$ la dimensione della tabella.

6.2 Tavole o Tabelle di hash

Le tabelle hash rappresentano un'evoluzione delle tabelle ad indirizzamento diretto, utilizzando una funzione di hash per mappare le chiavi alle posizioni dell'array. Se con l'indirizzamento diretto un elemento con chiave k è memorizzato nella cella k, con l'hashing, questo elemento è memorizzato nella cella h(k), ovvero si utilizza una funzione di hash **h** per calcolare la cella della chiave k. Dunque la funzione h associa ogni chiave k a una posizione h(k) nell'array.



La funzione hash $h : U \to \{0, 1, ..., m-1\}$ associa l'universo U delle possibili chiavi alle celle di una tavola di hash $T[0 ... m-1]$ dove la dimensione m della tavola è generalmente più piccola di $|U|$, in questo modo la dimensione dell'array viene ridotta e sarà proporzionale al numero massimo di chiavi presenti nel dizionario. Tuttavia, essendo $|U| > m$, si verificano **collisioni** quando due chiavi diverse k_1 e k_2 vengono mappate alla stessa posizione $h(k_1) = h(k_2)$.

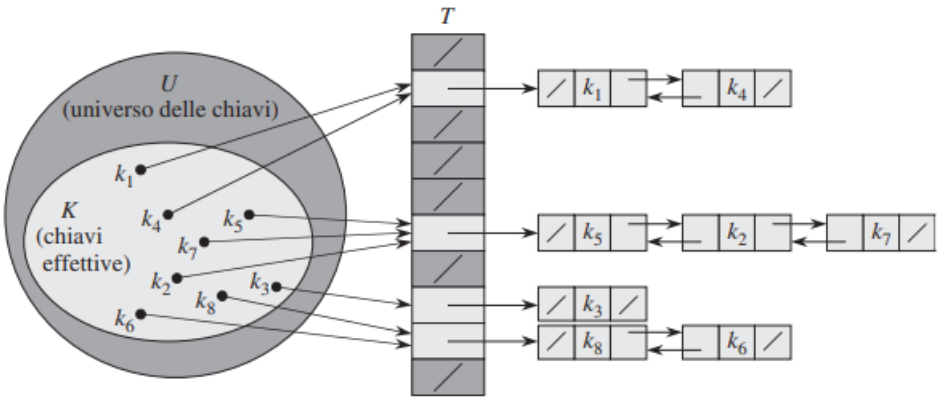
Ricorda che il motivo della collisione può essere spiegato con il **"principio della piccionaia"**: se p piccioni devono trovare posto in c caselle e ci sono più piccioni che caselle $(p > c)$ allora in qualche casella entreranno almeno due piccioni.

Gestione delle collisioni

La gestione delle collisioni è cruciale per l'efficienza delle tabelle hash. Esistono due metodi principali:

- Metodo del concatenamento (Liste di trabocco);
- Metodo dell'indirizzamento aperto.

6.2.1 Metodo del concatenamento (Liste di trabocco)



L'insieme di input di n elementi viene diviso casualmente in m sottoinsiemi, ciascuno di cardinalità n/m. Una funzione di hash determina a quale sottoinsieme appartiene un elemento, ovvero associa ad ogni elemento una chiave, per gestire i vari sottoinsiemi (chiavi) si realizzano delle **liste collegate**. Immaginiamo di avere più chiavi $k_1 \neq k_2 \neq k_3$ mappate su una stessa cella j dell'array T, questa ha un puntatore alla testa della lista concatenata che conterrà tutti gli elementi memorizzati in j, se non ci sono elementi, la cella j contiene la costante NIL. Per concatenare le liste associate a chiavi mappate alla stessa posizione si usufruisce di un doppio puntatore.

6.2.2 Operazioni sulle liste di trabocco

Le operazioni su una tabella con liste di trabocco sono le seguenti:

- **Search(T, k):** cerca nella lista $T[h(k)]$ un elemento x tale che $x.key == k$ e lo restituisce;
- **Insert(T, x):** aggiunge x nella lista $T[h(x.key)]$;
- **Delete(T, x):** toglie x dalla lista $T[h(x.key)]$.

6.2.3 Fattore di carico

Il concetto di fattore di carico α per il metodo del concatenamento può essere ridefinito come segue: data una tavola hash T con m celle dove sono memorizzati n elementi, si definisce fattore di carico α come il rapporto n/m, ossia il **numero medio di elementi memorizzati in una lista**.

Analisi

L'analisi sarà fatta in funzione del fattore di carico α, che può essere minore, uguale o maggiore di 1. Il caso peggiore si presenta quando tutte le n chiavi sono associate alla stessa cella, così creando una lista di lunghezza n con operazioni che richiedono il seguente tempo di esecuzione:

- **Search (T, k): $\Theta(n)$** + tempo di calcolo della funzione di hash;
- **Insert (T, x): $O(1)$** con l'assunzione che x non sia già nel dizionario, se questa assunzione non vale allora si paga il costo di una precedente search;
- **Delete (T, x): $O(1)$** se le liste sono doppiamente concatenate, ovvero quando ogni nodo della lista ha un campo *next* che punta al nodo successivo e un campo *prev* che punta al nodo precedente, altrimenti il costo di una search.

Le prestazioni dell'hashing nel caso medio dipendono dal modo in cui la funzione hash h distribuisce l'insieme delle chiavi da memorizzare tra le m celle. Supponiamo che qualsiasi elemento abbia la stessa probabilità di essere mandato in una qualsiasi delle m celle, indipendentemente dalle celle in cui sono mandati gli altri elementi, si tratta così l'ipotesi di hash uniforme indipendente.

6.2.4 Teoremi metodo del concatenamento

Teorema

In una tabella hash le cui collisioni sono risolte tramite la tecnica del concatenamento, una ricerca senza successo richiede in media un tempo $\Theta(1+\alpha)$, nell'ipotesi di hashing uniforme e indipendente.

Dimostrazione.
Nell'ipotesi di hashing uniforme indipendente per $j = 0, 1, ..., m-1$, indicando con n_j la lunghezza della lista $T[j]$, avremo $n = n_0 + n_1 + ... + n_{m-1}$ (n_j è la lunghezza di ciascuna delle m liste), il valore atteso di n_j sarà

$$E[n_j] = \frac{1}{m}\sum_{i=0}^{m-1} n_i = \frac{n}{m} = \alpha$$

ovvero la lunghezza attesa di una lista.
Supponiamo che basti un tempo $O(1)$ per calcolare il valore di hash $h(k)$, il tempo atteso per ricercare senza successo una chiave k è il tempo atteso per svolgere le ricerche fino alla fine della lista $T[h(k)]$ (ovvero $\Theta(n_j)$), che ha una lunghezza attesa pari a $E[n_{h(k)}] = \alpha$. Quindi, il numero atteso di elementi esaminato in una ricerca senza successo è α e il tempo totale richiesto è $\Theta(1+\alpha)$. □

Teorema

In una tavola hash le cui collisioni sono risolte tramite la tecnica del concatenamento, una ricerca con successo richiede in media un tempo $\Theta(1+\alpha)$, nell'ipotesi di hashing uniforme e indipendente.

Dimostrazione.
Assumendo che ogni chiave presente nella tabella abbia la stessa probabilità di essere la chiave cercata, la ricerca di una chiave k presente nella tabella richiede il calcolo dell'indice $j = h(k)$, il test sulle chiavi che precedono k nella lista $T[j]$ e, infine, il test su k. In questo modo si arriverà a svolgere un numero di operazioni pari a 2 + numero di elementi che precedono k nella lista. Il numero di elementi esaminati durante una ricerca con successo di un elemento k è uno in più del numero di elementi che si trovano prima di k nella lista $T[j]$. Le chiavi che precedono k sono state inserite tutte dopo di k, perché i nuovi elementi vengono posti all'inizio della lista. Supponiamo che $k = k_i$ sia l'i-esima chiave inserita nella lista con $i = 1, 2, ..., n$, per

$j = i+1,\ 2,\ ...,\ n$ sia $\mathbf{X_{i,j}}$ la variabile casuale indicatrice che vale 1 se k_j viene inserita nella stessa lista di k e 0 altrimenti:

$$X_{i,j} = \begin{cases} 0 & se\ h(k_i) \neq h(k_j) \\ 1 & se\ h(k_i) = h(k_j) \end{cases}$$

Nell'ipotesi di hashing uniforme indipendente $E[X_{i,j}] = 1/m$. Dunque, il numero atteso di elementi esaminati in una ricerca con successo è:

$$\begin{aligned}
&\mathrm{E}\left[\frac{1}{n}\sum_{i=1}^{n}\left(1+\sum_{j=i+1}^{n}X_{ij}\right)\right] \\
&\quad= \frac{1}{n}\sum_{i=1}^{n}\left(1+\sum_{j=i+1}^{n}\mathrm{E}\left[X_{ij}\right]\right) \quad \text{(per la linearità del valore atteso)} \\
&\quad= \frac{1}{n}\sum_{i=1}^{n}\left(1+\sum_{j=i+1}^{n}\frac{1}{m}\right) \\
&\quad= 1+\frac{1}{nm}\sum_{i=1}^{n}(n-i) \\
&\quad= 1+\frac{1}{nm}\left(\sum_{i=1}^{n}n-\sum_{i=1}^{n}i\right) \\
&\quad= 1+\frac{1}{nm}\left(n^2-\frac{n(n+1)}{2}\right) \quad \text{(per l'equazione (A.1))} \\
&\quad= 1+\frac{n-1}{2m} \\
&\quad= 1+\frac{\alpha}{2}-\frac{\alpha}{2n}
\end{aligned}$$

Se il numero n di elementi nella tabella è proporzionale al numero di celle m, ossia $n = c \cdot m$ per qualche costante c, abbiamo

$$\alpha = n/m = c \cdot m/m = costante$$

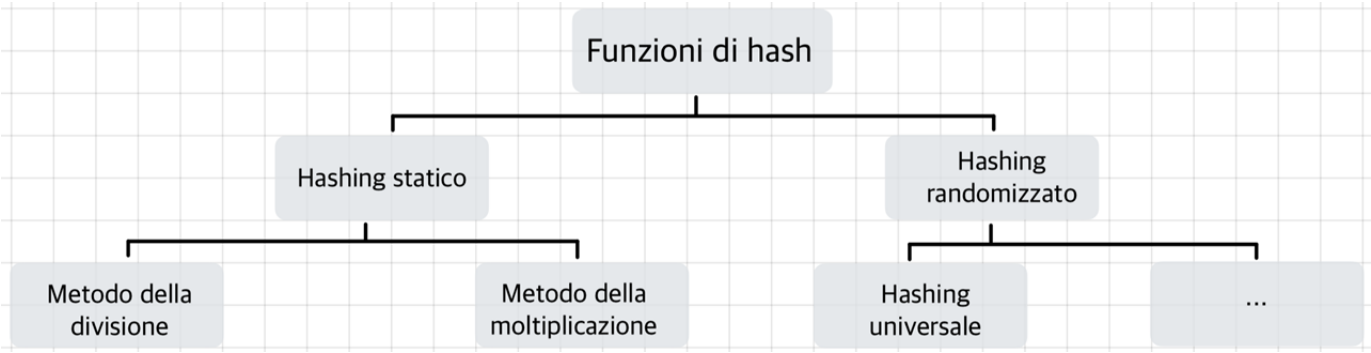
Pertanto la ricerca richiede in media un tempo costante e il suo costo è $\Theta(1+\alpha) = \Theta(1)$. Poiché l'inserimento e la cancellazione richiedono, nel caso peggiore, tempo $\Theta(1)$ quando le liste sono doppiamente concatenate, tutte le operazioni di dizionario possono essere svolte, in media, nel tempo $\Theta(1)$. □

Corollario

Utilizzando l'hashing universale e risolvendo le collisioni tramite concatenamento in una tabella inizialmente vuota di m celle, il tempo atteso per gestire qualsiasi sequenza di s operazioni Insert, Search e Delete contenente $n = O(m)$ operazioni Insert $\Theta(s)$.

Dimostrazione.
Sapendo che usando una funzione di hash universale si ha una probabilità di collisione tra due chiavi di al più $1/m$ che porta ad avere operazioni di costo medio costante. □

6.2.5 Funzioni di hash



Esistono degli schemi per poter progettare e realizzare delle buone funzioni di hash, due sono di tipo euristico, ovvero l'hashing per divisione e quello per moltiplicazione, mentre il terzo usa la randomizzazione ed è denominato hashing universale. Una buona funzione di hash soddisfa **l'ipotesi dell'hashing uniforme indipendente**: ogni chiave ha la stessa probabilità $1/m$ di essere mandata in una qualsiasi delle m celle, indipendentemente dalle chiavi inserite precedentemente.
Se le chiavi sono rappresentate da numeri reali x estratti casualmente e indipendentemente da una distribuzione di probabilità uniforme nell'intervallo $0 \leq k < 1$, la funzione di hash $h(x) = \lfloor m \cdot x \rfloor$ soddisfa l'ipotesi di hash uniforme indipendente.
Poiché questa ipotesi dipende dalle probabilità con cui vengono estratti gli elementi da inserire e, questa probabilità in genere non è nota, si assume che l'universo delle chiavi sia l'insieme dei numeri naturali interi non negativi. Questa assunzione non è restrittiva, in quanto ogni tipo di chiave è rappresentato nel calcolatore con una sequenza di bit e ognuna di esse si può interpretare come un intero non negativo.

6.2.6 Hashing statico

- **Metodo della divisione.** Associa ad una chiave k una delle m celle prendendo il resto della divisione intera fra k e m, ovvero

  $$h(k) = k \ mod \ m$$

  Per esempio, se la tavola hash ha dimensione $m = 12$ e la chiave è $k = 100$, allora $h(k) = 4$. Quando si utilizza questo metodo vengono evitati certi valore di m, ad esempio m non dovrebbe essere una potenza di 2, perché se $m = 2 \cdot p$, allora $h(k)$ rappresenta proprio i p bit meno significativi di k. Una buona scelta per m è un numero primo non troppo vicino ad una potenza esatta di 2. Ad esempio $m = 701$ perché è un numero primo vicino a 2000/3, ma non a una potenza di 2.

- **Metodo della moltiplicazione.** Questo metodo si svolge in due passi, prima di moltiplica la chiave k per una costante A nell'intervallo $0 < A < 1$ e si estrae la parte frazionaria di $k \cdot A$. Dopodiché si moltiplica il valore ottenuto per m e si prende la parte intera inferiore del risultato. La funzione di hash è

  $$h(k) = \lfloor m \cdot (k \ A \ mod \ 1) \rfloor$$

  dove "$k \ A \ mod \ 1$" rappresenta la parte frazionaria di $k \cdot A$, cioè $k \cdot A \ - \ \lfloor k \cdot A \rfloor$. Un vantaggio del metodo della moltiplicazione è che il valore di m non è critico.
  Tipicamente, si sceglie m pari ad una potenza di 2 ($m = 2 \cdot p$ per qualche intero p).

Supponiamo che ogni chiave k possa essere rappresentata da una sola parola di w bit. Come A prendiamo una frazione della forma $q/2w$, dove q è un intero nell'intervallo $0 < q < 2 \cdot w$. Prima si moltiplica k per l'intero di w bit $q = A \cdot 2 \cdot w$. Il risultato è un valore di $2 \cdot w$ bit $r_1\ 2^w + r_0$ dove r_1 è la parte più significativa del prodotto e r_0 è la parte meno significativa del prodotto. Il valore hash desiderato di p bit è formato dai p bit più significativi di r_0.

6.2.7 Hashing randomizzato

Nessuna funzione di hash può evitare che un avversario malevolo inserisca nella tabella una sequenza di valori che vengono mandati tutte alla stessa cella, ovvero nella stessa lista, generando un tempo medio di ricerca pari a $\Theta(n)$. Più in generale, qualsiasi funzione hash è vulnerabile a un comportamento così inefficiente nel caso peggiore. L'unico sistema efficace per migliorare la situazione è scegliere casualmente la funzione di hash in modo che sia indipendente dall'input, ovvero dalle chiavi che devono essere effettivamente memorizzate. Questo approccio, detto **hashing universale**, permette di scegliere all'inizio dell'esecuzione dell'algoritmo la funzione di hash casualmente da una classe di funzioni.

Definizione di "hashing universale". Un insieme H di funzioni di hash che associano un dato universo U di chiavi all'insieme $\{0,\ 1,\ ...,\ m-1\}$ degli indici della tabella hash si dice universale se, per ogni coppia di chiavi diverse j e k vi sono al più $|H|/m$ funzioni di hash in H tali che $h(j) = h(k)$.

In altre parole, con una funzione di hash scelta a caso da H, la probabilità di collisione fra due chiavi distinte j e k non è maggiore della probabilità $1/m$ di una collisione nel caso in cui $h(k)$ e $h(j)$ fossero scelte in modo casuale e indipendente dall'insieme degli indici della tabella hash (hashing universale indipendente).

6.2.8 Progettare una famiglia universale di funzioni hash – Metodo basato sulla teoria dei numeri

Iniziamo scegliendo un numero primo p sufficientemente grande in modo che sia maggiore di ogni possibile chiave k. Supponendo che la dimensione dell'universo delle chiavi sia maggiore del numero di celle nella tavola hash, abbiamo $p > m$. Adesso definiamo la funzione hash $h_{a,b}$ per ogni coppia di interi $(a,\ b)$ tali che $1 \leq a < p$ e $0 \leq b < p$ come segue, ovvero utilizzando una trasformazione lineare seguita da riduzioni modulo p e poi modulo m:

$$h_{a,b}(k) = [(ak + b)\ mod\ p]\ mod\ m$$

La famiglia di tutte queste funzioni hash è

$$H_{pm} = \{h_{a,b} :\ 1 \leq a < p,\ 0 \leq b < p\}$$

con p numero primo, ed è universale.

Teorema

La famiglia di funzioni $H = \{h_a :\ a\ è\ dispari,\ 1 < q < m\}$ dove $h_a(k) = (ka\ mod\ 2^w)$ ottenuta tramite metodo moltiplica e scorri è 2/m-universale.

6.3 Metodo dell'indirizzamento aperto

Nell'indirizzamento aperto, tutti gli elementi sono memorizzati nella tavola hash stessa, ovvero ogni cella della tavola contiene un elemento dell'insieme dinamico o la costante NIL. Non si utilizzano puntatori a zone esterne di memoria, motivo per il quale un inserimento potrebbe non andare a buon fine a causa del fatto che la tabella potrebbe essere piena, perciò il fattore di carica α non supera mai 1.
Per effettuare un inserimento si esaminano in successione le posizioni della tavola hash (ispezione), finché non si trova una cella vuota in cui inserire la chiave. Anziché seguire sempre lo stesso ordine 0, 1, ..., m-1, la sequenza delle posizioni esaminate durante un'ispezione dipende dalla chiave da inserire. Per tal motivo, la funzione di hash è una funzione $h(k,\ i)$ che al variare di i tra 0 e m-1 fornisce, per ciascuna chiave k, una sequenza di indici $\{h(k,0),\ h(k,1),\ ...,\ h(k,m-1)\}$ che rappresenta l'ordine di ispezione. Volendo poter ispezionare tutte le celle, la sequenza deve essere una permutazione dell'insieme degli indici della tabella.

Ricorda: nello pseudocodice di Insert si suppone che gli elementi della tavola hash T siano chiavi senza dati satelliti, ovvero la chiave k è identica all'elemento che contiene la chiave k. Ogni cella contiene una chiave o la costante NIL (se la cella è vuota).

Ricorda: quando cancelliamo una chiave dalla cella i, non possiamo semplicemente marcare questa cella come vuota inserendovi la costante NIL. Così facendo, potrebbero essere interrotte le sequenze di ispezione. Una soluzione consiste nel marcare la cella registrandovi il valore speciale **DELETED**, anziché NIL. Di conseguenza è necessario adattare la funzione Insert.

Pseudo Codice

```
Insert (T, k){
    i = 0
    while i != m
        j = h(k, i)
        if T[j] == nil or T[j] == DELETED
            T[j] = k
            return j
        else
            i = i + 1

    return "tabella piena"
}
Search(T, k){
    i = 0
    while i != m and T[q] != nil
        q = h(k, i)
        if T[q] == k
            return q
        i = i + 1

    return nil
}
Delete(T, i){
    T[i] = DELETED
}
```

Con l'indirizzamento aperto, la funzione di hash fornisce una sequenza di ispezione, pertanto si modifica l'ipotesi di hashing indipendente e uniforme e si parla di hashing con permutazioni indipendenti e uniformi. Per **l'hashing uniforme** si suppone che ogni chiave abbia la stessa probabilità $1/m!$ di generare una qualsiasi delle $m!$ possibili sequenze di ispezione.
Esistono tre tecniche comunemente utilizzate per calcolare le sequenze di ispezione richieste dall'indirizzamento aperto:

- ispezione lineare
- ispezione quadratica
- doppio hashing

Nessuna delle tre genera tutte le $m!$ sequenze di ispezione, le prime due ne generano soltanto m e l'ultima ne genera m^2.

6.3.1 Ispezione lineare

La funzione di hash $h(k, i)$ si ottiene da una funzione di hash ordinaria $h'(k)$ ponendo

$$h(k, i) = [h'(k) + i] \ mod \ m$$

L'esplorazione inizia dalla cella $h(k, 0) = h'(k)$ e continua con le celle $h'(k)+1,\ h'(k)+2,\ etc...$ fino ad arrivare alla cella m-1, dopodiché si continua con le celle 0, 1, ecc. fino ad aver percorso circolarmente tutta la tabella. L'ispezione lineare è facile da implementare ma soffre del problema noto come **addensamento primario (clustering)**: si formano lunghe file di celle occupate, che aumentano il tempo medio di ricerca. Ciò significa che i nuovi elementi inseriti nella tabella tendono ad addensarsi attorno agli elementi già presenti.

6.3.2 Ispezione quadratica

La funzione di hash $h(k, i)$ si ottiene da una funzione di hash ordinaria $h'(k)$ ponendo

$$h(k, i) = [h'(k) + c_1 i + c_2 i^2] \ mod \ m$$

dove c_1 e c_2 sono due costanti con $c_2 \neq 0$. I valori di m, c_1 e c_2 non possono essere qualsiasi ma devono essere scelti opportunamente in modo che la sequenza di ispezione percorra tutta la tabella.

6.3.3 Ispezione con doppio hashing

La funzione di hash $h(k, i)$ si ottiene da due funzioni di hash ordinarie $h_1(k)$ e $h_2(k)$ ponendo

$$h(k, i) = [h_1(k) + i h_2(k)] \ mod \ m$$

Perché la sequenza di ispezione percorra tutta la tabella, il valore di $h_2(k)$ deve essere relativamente primo con m. È possibile soddisfare questa condizione in diversi modi:

1. Scegliendo $m = 2p$ e definendo $h_2(k) = 2h'(k)+1$ in modo che produca sempre un numero dispari, con $h'(k)$ funzione di hash qualsiasi per una tabella di dimensione $m' = m/2 = 2p - 1$;
2. Scegliendo m primo e definendo $h_2(k)$ in modo che generi sempre un numero intero positivo minore di m. Ad esempio, $h_1(k) = k \ mod \ m$, $h_2(k) = 1 + (k \ mod \ m')$, dove m' è minore di m, di solito $m' = m - 1$.

7 Altri Algoritmi

7.1 Problema della Selezione

Il problema della selezione è definito da un input e da un output ed è definito come segue:

Input: Un insieme A di n elementi (distinti) rappresentati da un array ed un valore intero k, con $1 \leq k \leq n$, che rappresenta la posizione nell'array.

Output: L'elemento $x \in A$ tale che è il k-esimo elemento più piccolo di A oppure k-esima statistica d'ordine di A.
Per esempio, il minimo di A è la prima statistica d'ordine con k = 1, mentre il massimo è l'n-esima statistica d'ordine k = n.

Definizione. La mediana è l'elemento che si trova al centro della collezione:

1. Se n è dispari la mediana è unica e il suo indice è pari a $\frac{n+1}{2} = \lceil \frac{n}{2} \rceil$
2. Se n è pari ci sono due mediane: la mediana inferiore di indice $\frac{n+1}{2} = \lfloor \frac{n}{2} \rfloor$ e la mediana superiore di indice $\frac{n+1}{2} = \lceil \frac{n}{2} \rceil$.

Approfondiamo adesso il problema di selezione nella ricerca del minimo e del massimo. Per cercare il min o il max siamo abituati a svolgere n confronti, ovvero confrontare ciascun elemento dell'array con il min e il max, è possibile svolgere meno confronti? Come sappiamo il costo asintotico nel caso peggiore per la ricerca del min o max è $\Theta(n)$, non è possibile trovare un algoritmo che faccia meglio infatti il min o il max di una collezione A sono delle proprietà globali, quindi, necessariamente bisognerà confrontare ogni elemento dell'array. Nonostante ciò, è comunque possibile diminuire il numero di confronti, infatti anzi di scansionare un solo elemento per ciclo è possibile scansionarli a coppie così che il numero di cicli passi da n a $\frac{n}{2}$. Il nome di questo algoritmo è Min_Max.

7.1.1 PseudoCodice Min_Max

```
}
Min_Max(A, n){
    if(n e' dispari){
        max= A[1]
        min= A[1]
        i= 2
    }
    else{
        if(A[1] < A[2]){
            max= A[2]
            min= A[1]
        }
        else{
            max= A[1]
            min= A[2]
        }
        i= 3
    }
```

```
while(i < n){
    if (A[i] < A[i+1]){
        if A[i] < min
            min= A[i]
        if A[i+1] > max
            max= A[i+1]
    }
    else{
        if A[i+1] < min
            min= A[i+1]
        if A[i] > max
            max= A[i]
    }
    i= i+2
}
return Min_Max // non e' una chiamata ricorsiva, indica semplicemente di restituire il max e il m
```

Esempio: $A =< 5, 8, 23, 1, 7, 9 >$ con n= 6

Passo 1
n è pari allora prendo i primi due valori dell'array e li confronto e assegno min= 5 e max= 8 e pongo i= 3

Passo 2
Iterazione 1:

Prendo la coppia A[i]= 23 e A[i+1]= 1:
A[i] < A[i+1] ? No, allora:
    A[i+1] < min= 5 ? Si, allora min= 1
    A[i] > max= 8 ? Si, allora max= 23
i= 3+2= 5

Iterazione 2:

Prendo la coppia A[i]= 7 e A[i+1]= 9
A[i] < A[i+1] ? Si, allora:
    A[i] < min= 1 ? No
    A[i+1] > max= 23 ? No
i= 5+2= 7

Esco dal ciclo e il programma termina con min= 1 e max= 23

Costo misurato in termini di numero di confronti

	n dispari	**n pari**
Init	1 confronto → 0 confronti	2 confronti → 1 confronto
Iterazione 1	confronti	3 confronti
Iterazione 2	3 confronti	3 confronti

Nell'inizializzazione vengono effettuati un confronto nel caso n sia dispari e, due nel caso sia pari. Notiamo come il primo controllo, ovvero se n è pari o dispari, viene svolto a prescindere quindi è possibile semplificare il numero di confronti dell'init così da semplificarci i calcoli. Nelle iterazioni del ciclo while vengono effettuati sempre 3 confronti, allora è possibile dire che

il numero di confronti se n è pari o dispari è:

n dispari: $3 \cdot (\frac{n-1}{2})$

n pari: $3 \cdot \left(\frac{n-2}{2}\right) + 1$

Analizziamo i due casi. Nel caso di **n dispari** viene assegnato ad i il valore 2, questo significa che il while farà $\frac{n-1}{2}$ cicli; infatti, bisogna notare come i venga incrementato di 2 ogni volta, dimezzando così il numero di cicli. Il tutto è moltiplicato per 3 in quanto come abbiamo visto il numero di confronti per ciclo è 3. Nel caso di **n pari** dobbiamo considerare il costo costante del confronto (+1) e notare in seguito come ad i venga assegnato il valore 3. Questo significa che, ragionando come prima, il numero di cicli che effettuerà il while sarà $\frac{n-2}{2}$, il tutto moltiplicato per 3 dato il numero di confronti per ciclo. Si può notare come $\frac{n-1}{2}$ sia uguale a $\lfloor \frac{n}{2} \rfloor$ allora il costo per **n dispari** è $3 \cdot \lfloor \frac{n}{2} \rfloor$. Nel caso di **n pari**, sviluppando i calcoli si ottiene:

$$3 \cdot \left(\frac{n-2}{2}\right) + 1 = 3 \cdot \frac{n}{2} - 3 + 1 = 3 \cdot \frac{n}{2} - 2 < \ 3 \cdot \lfloor \frac{n}{2} \rfloor$$

Allora nel caso asintotico il numero di confronti che vengono effettuati sia nel caso di **n pari** sia nel caso di **n dispari** equivale a $3 \cdot \lfloor \frac{n}{2} \rfloor$.

Ricorda: Il costo computazionale asintotico è sempre un costo lineare quindi $\Theta(n)$, quello che cambia sono esclusivamente le performance dell'algoritmo in quanto diminuisce il numero di confronti che dovranno essere svolti.

7.2 Selezione in tempo atteso lineare

7.2.1 PseudoCodice

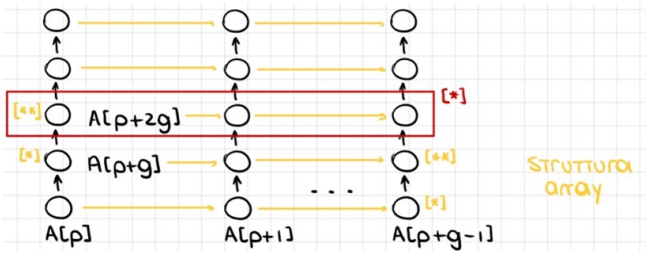
```
Select(A, p, r, i){
    while (r−p+1)mod 5 != 0  //operazione di modulo ovvero il resto
        for j= p+1 to r
            if A[p] > A[j]
                Scambia A[p] e A[j]
        if i==1
            return A[p]
        p= p+1
        i= i+1
    g= (r−p+1)/5  // numero di cinquine su cui andro' a lavorare
    for j= p to p+g+1
        ordina la cinquina
        A[j] .. A[j+4]

    // richiamo ricorsivamente sulle mediane delle cinquine e restituisce la mediana delle mediane
    x= Select( A, p+2g, p+3g−1, (g+1)/2)
    // (g+1)/2 e' equivalente all'approsimazione superiore di g/2

    q= partiziona( A, p, r, x) // Crea due sottoarray e restituisce il pivot
    k= q−p+1
    if i==k
        return A[q]
    else if i<k
```

```
        //richiamo ricorsivamente nella parte sinistra del pivot
        return Select(A, p, q−1, i)
    else
        //richiamo ricorsivamente nella parte destra del pivot scalando l'indice i
        return Select(A, p, q−1, i−k)
}
```

7.2.2 Analisi



Le cinquine sono costruite verticalmente, in modo che gli elementi che le costituiscono siano vicini tra loro. Di ciascuna si seleziona la mediana, con la caratteristica di essere maggiore degli elementi precedenti e minore dei successivi della cinquina. In questo modo si è messo in evidenza l'insieme delle mediane (evidenziato in rosso nel disegno). Con l'istruzione $x = select(A, p{+}2g, p{+}3g{-}1, \lceil \frac{g}{2} \rceil)$ si cerca la mediana delle mediane all'interno dell'insieme, questo coincide con l'elemento al centro dell'array complessivo. Una volta chiamata la funzione **partiziona** questa sposta tutti gli elementi minori di x nel sottoarray di sinistra e quelli maggiori nel sottoarray di destra. Se i==k si è trovata la k-esima statistica d'ordine, altrimenti si chiama ricorsivamente la funzione **select** sul sottoarray di sinistra se $i < k$, o su quello di destra.

7.3 Problema delle inversioni

Definizione. Dato un array A[1:n] di elementi distinti, se $i < j$ e $A[i] > A[j]$ allora si dice che la coppia (i, j) è un'inversione di A.

Esempio – Dato il seguente array $A =< 2, 3, 8, 6, 1 >$ contare il numero di inversioni.
Inversioni:

1. (0, 4), infatti $0 < 4$ e $2 > 1$
2. (1, 5)
3. (2, 3)
4. (2, 4)
5. (3, 4)

Il caso peggiore è quando l'array è ordinato al contrario infatti per ogni elemento conterò $n{-}1{-}i$ inversioni (i è la posizione dell'elemento i-esimo). Quindi avrò:

$$(n-1) + (n-2) + (n-3) + ... + 1 = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} \in O(n^2)$$

8 Altro

8.1 Modello di Calcolo

Per implementare i nostri algoritmi dobbiamo aver in mente su quale macchina questi verranno adoperati. In particolare, in questo corso faremo riferimento ad un generico sistema di calcolo noto come RAM (Random Access Machine) a costi uniformi. La RAM è un modello in cui le istruzioni sono eseguite una dopo l'altra, senza operazioni contemporanee. In particolare, consideriamo un modello con memoria grande a piacere, questa è suddivisa in tante celle, ad ognuna di esse è associato un indirizzo di memoria e può contenere un qualsiasi numero intero. Parlare invece di costi uniformi vale a dire che si assume che ogni operazione abbia costo unitario in relazione al tempo e alla memoria utilizzata.